



Development and Evaluation of Automatic Passenger Counting Algorithms based on 2D Video Data Combining LSTM and U-Net

Thesis in partial fulfillment of the requirements for the degree
Master of Science (M.Sc.)

submitted by
Mohamed Matar

Matriculation number
457204

Under the scientific supervision of
Prof. Dr. Klaus Obermayer

Under the scientific guidance of
Dr. Sambu Seo

Berlin, November 2024

Technische Universität Berlin
Fakultät IV – Elektrotechnik und Informatik
Institut for Software Engineering and Theoretical Computer Science

Affidavit

- ✓ Die Arbeit enthält eine deutsche Zusammenfassung.
- ✓ Die Arbeit enthält eine englische Zusammenfassung.
- ✓ Die Arbeit enthält eine Selbständigkeitserklärung nach § 60 Abs. 8 AllgStu-PO:

Hiermit erkläre ich an Eides statt, dass ich die vorliegende Arbeit selbstständig und eigenhändig sowie ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe.

MOHAMED MATAR
Berlin, 16. März 2025

- ✓ The thesis contains a German abstract.
- ✓ The thesis contains an English abstract.
- ✓ The thesis includes a declaration that I completed the work independently in accordance with section § 60 Abs. 8 AllgStuPO:

I hereby confirm that I prepared this thesis independently and by exclusive reliance on literature or tools indicated herein.

MOHAMED MATAR
Berlin, March 16, 2025

Abstract

Deutsch

In öffentlichen Nahverkehrssystemen ist eine genaue Fahrgastzählung von entscheidender Bedeutung, um eine effiziente Verteilung der Fahrgeldeinnahmen zu gewährleisten, das Auslastungsmanagement zu optimieren und die Gesamteffizienz des Verkehrsnetzes zu verbessern.

In dieser Arbeit wird der Einsatz von Deep-Learning-Techniken, insbesondere von U-Net- und LSTM-Architekturen, untersucht, um die Aufgabe der automatischen Fahrgastzählung im öffentlichen Nahverkehr zu bewältigen. Der kombinierte Einsatz von U-Net und LSTM nutzt die Stärken sowohl der räumlichen Merkmalsextraktion als auch der zeitlichen Sequenzanalyse, um die Genauigkeit der Fahrgastzählung zu verbessern.

Unsere Ergebnisse zeigen, dass das vorgeschlagene Modell eine hohe Leistung erzielen kann, wobei die beste Konfiguration eine Genauigkeit von $94,65\% \pm 0,5\%$, eine ausgeglichene Genauigkeit von $74,03\% \pm 1,04\%$ und eine relative globale Verzerrung (RGB) von $-0,89\% \pm 0,44\%$ erreicht. Diese Ergebnisse deuten darauf hin, dass die Kombination aus U-Net und LSTM für die Aufgabe der automatischen Fahrgastzählung effektiv ist und eine zuverlässige Lösung für öffentliche Nahverkehrssysteme darstellen kann.

Schlüsselwörter: *U-Net, LSTM, Automatische Fahrgastzählung*

English

In public urban transportation systems, accurate passenger counting is crucial for ensuring the efficient distribution of ticket revenues, optimizing load management, and improving the overall effectiveness of the transportation network.

This thesis investigates the use of deep learning techniques, specifically the U-Net and LSTM architectures, to tackle the task of automatic passenger counting in urban transportation. The combined use of U-Net and LSTM leverages the strengths of both spatial feature extraction and temporal sequence analysis, aiming to improve accuracy in passenger counting.

Our findings demonstrate that the proposed model can achieve high performance, with the best configuration reaching an accuracy of $94.65\% \pm 0.5\%$, a balanced accuracy of $74.03\% \pm 1.04\%$, and a relative global bias (RGB) of $-0.89\% \pm 0.44\%$. These results suggest that the combination of U-Net and LSTM is effective for the task of automatic passenger counting and can be a reliable solution for public urban transportation systems.

Keywords: *U-Net, LSTM, Automatic Passenger Counting System*

Contents

List of Abbreviations	1
1 Introduction	2
2 Theoretical Background	4
2.1 Neural Networks	4
2.2 Recurrent Neural Network	5
2.2.1 Backpropagation Through Time	8
2.2.2 Exploding/Vanishing Gradient Problem	10
2.3 Long Short-Term Memory	11
2.3.1 Architecture	12
2.3.2 Addressing the Vanishing and Exploding Gradients	14
2.4 Convolutional Neural Network	14
2.4.1 Convolution	15
2.4.1.1 Padding	17
2.4.1.2 Strides	18
2.4.2 ReLU Activation Function	18
2.4.3 Pooling	19
2.4.4 Up-Convolutions (Transposed Convolutions)	21
2.5 U-Net	22
3 Data	24
3.1 Origin	24
3.2 Annotations	25
3.3 Collection Conditions	26
3.4 Distributions	27
3.5 Preprocessing	28
3.6 Split	30
3.7 Augmentation	30
4 Methods	32
4.1 Implementation	32
4.1.1 U-Net	32
4.1.2 LSTM	34

4.2	Cost Function	35
4.2.1	Primary Loss	35
4.2.2	Auxiliary Losses	36
4.3	Reset On Stall	37
4.4	Performance Measures	38
4.4.1	Accuracy	38
4.4.2	Balanced Accuracy	39
4.4.3	Relative Global Bias	40
4.5	Batch Size	40
5	Experiments	41
5.1	Control Experiment - LSTM Only	42
5.2	U-Net with Varying Encoder/Decoder Blocks	44
5.2.1	3 Encoder Blocks	45
5.2.1.1	Dataset A	45
5.2.1.2	Dataset B	46
5.2.1.3	Dataset C	47
5.2.2	5 Encoder Blocks	48
5.2.2.1	Dataset A	48
5.2.2.2	Dataset B	50
5.3	U-Net with Batch Normalization	51
5.3.1	Dataset A	52
5.3.2	Dataset B	52
5.3.3	Dataset C	54
5.4	CNN and LSTM	55
6	Results	57
6.1	Control Experiment - LSTM Only	57
6.2	U-Net with Varying Encoder/Decoder Blocks	57
6.3	U-Net with Batch Normalization	58
6.4	Simplified CNN and LSTM Combination	58
6.5	Comparison to other Studies	59
6.6	Additional Observations	59
7	Discussion	60
8	Conclusion	62
	List of References	64
A	Model Performances in Numbers	67
A.1	LSTM Only	67

CONTENTS

A.2	U-Net with Varying Encoder/Decoder Blocks	68
A.2.1	3 Encoder Blocks	68
A.2.1.1	Dataset A	68
A.2.1.2	Dataset B	69
A.2.1.3	Dataset C	69
A.2.2	5 Encoder Blocks	70
A.2.2.1	Dataset A	70
A.2.2.2	Dataset B	71
A.3	U-Net with Batch Normalisation	72
A.3.1	Dataset A	72
A.3.2	Dataset B	73
A.3.3	Dataset C	73
A.4	CNN and LSTM	74
B	Failed Experiments	75
B.1	U-Net with Varying Encoder/Decoder Blocks	75
B.1.1	3 Encoder Blocks	75
B.1.1.1	Dataset A	75
B.1.1.2	Dataset B	75
B.1.2	5 Encoder Blocks	76
B.1.2.1	Dataset A	76
B.1.2.2	Dataset B	76

List of Abbreviations

APCS	Automatic Passenger Counting System
BPTT	Backpropagation Through Time
CNN	Convolutional Neural Network
HPC	High Performance Cluster
LSTM	Long Short-Term Memory
PUT	Public Urban Transportation
ReLU	Rectified Linear Unit
RGB	Relative Global Bias
RNN	Recurrent Neural Network

1 Introduction

Counting the number of passengers in a *Public Urban Transportation* (PUT) vehicle has been a task for PUT companies since their founding. Knowing the number of passengers using each PUT stop is crucial to determine the number of vehicles sent to each location and to balance the system's load.

Modern PUT systems often sell tickets based on regions and not on service providers. In Berlin, tickets bought from BVG are eligible for trains provided by the BVG itself, and every Deutsche Bahn train driving within Berlin. This leads to another need for accurate passenger counting systems. The distribution of ticket sale profits to each service provider is dependent on the passengers' vehicle usage.

Passenger Counting Systems started simple, with humans manually counting passengers at defined locations. Over time, *Automatic Passenger Counting Systems* (APCSs) were introduced to reduce human error and increase the granularity of the data. There are many ways to solve this task, including integrated systems such as RFID tag checks or gate-based systems, as well as independent systems such as pressure sensors, Wi-Fi-based systems, and computer vision systems.

Each system is integrated into an already existing PUT infrastructure, which varies from one provider to another. Therefore system need to be compatible with various PUT systems (Grgurević et al., 2022).

This thesis explores the use of U-Net Models together with LSTM to count the number of passengers entering or exiting PUT vehicles based on 2D video data recorded by security cameras. This provides a cost-efficient solution to the APCS problem, as most PUT vehicles are already equipped with security cameras.

The thesis was written in collaboration with Interautomation Deutschland GmbH, which provided the data for the experiments. Previous research has successfully tested the use of LSTM in APCSs. Our goal is to extend this by adding U-Net.

U-Net Models are mainly used in image segmentation tasks. In our experiments, we explore using U-Net to meaningfully preprocess the video data to improve the model.

This thesis will provide an introduction to the components of both LSTM and U-Net. We then describe the dataset and analyze its characteristics, after which the experimental setup will be described. Four experiments with different hyperparameters and configurations will be presented. The results of our experiments are summarized, and a discussion about their shortcomings is added. Finally, a conclusion about whether or not to use such a system is drawn.

2 Theoretical Background

To understand the Model and its results, we must build a strong theoretical foundation for how each component works. We begin with a small introduction to neural networks, followed by an explanation of how sequential data can be processed using *Recurrent Neural Networks* (RNNs) to understand LSTM. Afterward, we will discuss *Convolutional Neural Networks* (CNNs) to understand U-Net.

2.1 Neural Networks

Neural Networks are a key concept in Machine Learning.

A neural network, in general, consists of multiple perceptrons connected to one another. Perceptrons are small units that take in input values, process them, and then return an output. A perceptron processes inputs by calculating a weighted sum of the input and adding a bias, followed by applying an activation function to determine the output. (Goodfellow et al., 2016, pp. 168–170)

Perceptrons are organized into layers within a neural network. The first layer, known as the input layer, serves as the starting point of the network and accepts input data in the form of vectors. Following the input layer, the network contains one or more hidden layers wherein each perceptron is connected by a weighted connection to other perceptrons, with each perceptron possessing a bias term (Nielsen, 2015, p. 13).

The final layer is the output layer, which produces results in a human-interpretable form, such as the predicted class for a classification task (Nielsen, 2015, p. 13).

During training, the neural network's weights are adjusted to minimize the cost function, which quantifies the error between the network's predictions and the

actual targets. This process involves backpropagation, where the error is propagated backward through the network to update the weights using gradient descent (Nielsen, 2015, p. 39).

Neural networks are commonly trained using one of three learning paradigms: supervised learning, unsupervised learning, and reinforcement learning.

In reinforcement learning, a model learns by interacting with an environment and receiving feedback in the form of rewards or penalties based on its actions (Sutton and Barto, 2018, p. 2). In unsupervised learning, the model observes several examples of a vector x and implicitly or explicitly learns its probability distribution $p(x)$ and identifies patterns and structures within an unlabeled dataset. In supervised learning, the model observes both a vector x and an associated vector y . It learns to predict y from x by estimating $p(y|x)$. Those associated vectors y are also understood as labels in a labeled dataset. The labeled training dataset is often referred to as the golden dataset and provides the ground truth and goal for the model. It is created by human annotators that manually classify each data point in a dataset. These labels act as instructors for the algorithm, guiding it through the training phase. (Goodfellow et al., 2016, p. 140).

Now that we have established a general understanding of how neural networks are constructed and how they function, we can delve into more specialized types of neural networks designed to address specific types of problems.

2.2 Recurrent Neural Network

Recurrent Neural Networks (RNNs) are a family of neural networks designed to process sequential data. They excel in tasks where the context and order of data points are crucial, such as natural language processing, time-series forecasting, and video analysis.

Traditional feedforward networks process fixed-length inputs to produce outputs. However, these networks cannot manage data of variable length, such as videos, texts, or time-series data. RNN addresses this limitation by allowing sequential data or even variable-length sequences to be processed. This is achieved through parameter sharing, where the same set of parameters (weights) are used across

different time steps, enabling the model to generalize and learn patterns over sequences of varying lengths (Goodfellow et al., 2016, p. 378).

In our case, we want to analyze a video sequence of passengers entering or exiting a vehicle and produce two final values: the count of passengers entering and of those leaving the vehicle. These models are called many-to-one sequence models (Goodfellow et al., 2016, p. 378).

To understand RNNs, let's start by examining a traditional feedforward network as illustrated in figure 2.1.

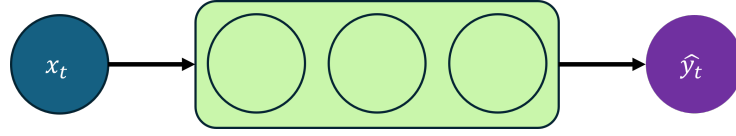


Fig. 2.1: Simple illustration of a feedforward network with an input x_t and a predicted output $\hat{y}_t$. The central circles represent layers with some perceptrons.

The input vector $x_t \in \mathbb{R}^m$ is transformed by some hidden layers to predict the output vector $\hat{y}_t \in \mathbb{R}^n$. Here, t denotes a single timestamp.

$$\hat{y}_t = f(x_t) \quad (2.1)$$

Our network learns the function f . With sequential data, this model would only use one timestamp x_t to predict y_t . Therefore, y would never rely on inputs from previous timestamps. To solve this, we can adjust the function to include information from a previous state.

$$\hat{y}_t = f(x_t, h_{t-1}) \quad (2.2)$$

The hidden state h can be thought of as the memory maintained by the neurons in the network. Now, $\hat{y}_2$ relies not only on x_2 but also on h_1 , the past memory.

In our case, since we are only interested in the final results and not the intermediate outputs $\hat{y}_0$ and $\hat{y}_1$, we can hide those for the final product. They are still computed, as the intermediate outputs improve the model's performance during the backward pass (Goodfellow et al., 2016, p. 375).

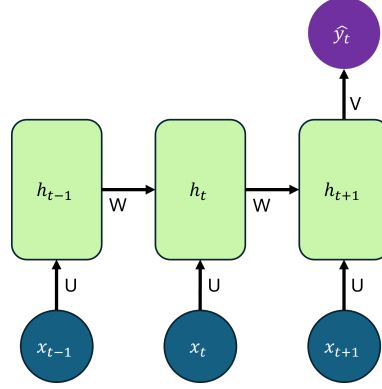


Fig. 2.2: Unrolled RNN.

In an RNN, the same weight matrices U , V , and W are used throughout the model. As illustrated in the unfolded RNN in figure 2.2, h values differ only in the computation time, which can also be depicted as a loop.

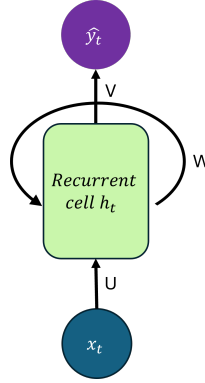


Fig. 2.3: Folded RNN.

In the recurrent cell, h_t is iteratively updated and fed back into the neuron's computation. Forward propagation begins with the initial state h_0 , and for each time step $t = 1$ to T , we apply the update equation:

$$h_t = \sigma(b + Wh_{t-1} + Ux_t) \quad (2.3)$$

Here, b is the bias term, and U and W are weight matrices for input-to-hidden and hidden-to-hidden connections, respectively. σ is the sigmoidal activation function.

Finally, the output can be produced using:

$$o_t = c + Vh_t \quad (2.4)$$

$$\hat{y}_t = \text{softmax}(o_t) \quad (2.5)$$

Where c is the bias vector, and V is the weight matrix for the hidden-to-output connection (Goodfellow et al., 2016, pp. 374–381).

The backward pass can be performed using either Real Time Recurrent Learning (RTRL) (Robinson and Fallside, 1987) or *Backpropagation Through Time* (BPTT) (Werbos, 1990). Since BPTT is simpler and faster, albeit more memory-intensive, we use BPTT (Nielsen, 2015, p. 39). In this method, individual losses across the steps in our sequence add up to the overall loss. Instead of backpropagating through a single weight matrix per step, we backpropagate the loss through each individual time step.

2.2.1 Backpropagation Through Time

Backpropagation Through Time (BPTT) is an extension of the standard backpropagation algorithm used to train RNNs. The goal of BPTT is to update each weight matrix U , V , and W by minimizing a loss function L that measures the difference between the predicted output and the true label.

To achieve this, we calculate the partial derivatives of L with respect to each weight matrix: $\frac{\partial L}{\partial U}$, $\frac{\partial L}{\partial V}$, and $\frac{\partial L}{\partial W}$. These derivatives are then used to update the weights using gradient descent, starting at the final layer and moving backward through the network. We update each weight matrix using the following:

$$W = W - \eta \frac{\partial L}{\partial W} \quad (2.6)$$

where η is the learning rate.

After the forward pass, the loss L is calculated based on the difference between the predicted output $\hat{y}$ and the true output y . This loss depends on the weight matrix V :

$$L \rightarrow \hat{y} \rightarrow V \quad (2.7)$$

The gradient of the loss with respect to V is similar to standard backpropagation in feedforward networks:

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial V} \quad (2.8)$$

The computation for W and U is more complex because these matrices are used at each time step throughout the model. For W , the gradient depends not only on the current hidden state h_t but also on all previous hidden states $h_{t-1}, h_{t-2}, \dots, h_0$:

$$\begin{aligned} L \rightarrow y \rightarrow h_t \rightarrow W \\ \hookrightarrow h_{t-1} \rightarrow W \end{aligned}$$

In BPTT, we compute the gradient of the weights by treating the weights at different time steps as distinct weights and then summing up the gradients:

$$\begin{aligned} \frac{\partial L}{\partial W_t} &= \frac{\partial L}{\partial y} * \frac{\partial y}{\partial h_t} * \frac{\partial h_t}{\partial W} \\ \frac{\partial L}{\partial W_{t-1}} &= \frac{\partial L}{\partial y} * \frac{\partial y}{\partial h_t} * \frac{\partial h_t}{\partial h_{t-1}} * \frac{\partial h_{t-1}}{\partial W} \end{aligned}$$

The total gradient with respect to W is obtained by summing the gradients over all time steps:

$$\frac{\partial L}{\partial W} = \sum_{t=0}^T \frac{\partial L}{\partial W_t} \quad (2.9)$$

Similarly, for U , the gradient is:

$$\frac{\partial L}{\partial U} = \sum_{t=0}^T \frac{\partial L}{\partial U_t} \quad (2.10)$$

BPTT allows RNNs to learn from sequential data by unrolling the network in time and applying backpropagation at each time step. This method is computationally intensive but essential in training RNN models (Goodfellow et al., 2016, pp. 383–386).

2.2.2 Exploding/Vanishing Gradient Problem

RNNs are prone to the vanishing and exploding gradient problem during training.

The vanishing gradient problem occurs because, in RNNs, gradients are propagated backward through many time steps. As a result, each time step incorporates a multiplicative factor due to the chain rule, often leading to gradients that shrink exponentially.

Consider the derivative of the sigmoidal activation function:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.11)$$

$$\sigma'(x) = \frac{d}{dx} \left(\frac{1}{1 + e^{-x}} \right) = \frac{e^{-x}}{(1 + e^{-x})^2} = \sigma(x)(1 - \sigma(x)) \quad (2.12)$$

The maximum value of $\sigma'(x)$ occurs when $\sigma(x) = 0.5$, yielding $\sigma'(x) = 0.25$. Therefore, the gradient $\frac{\partial L}{\partial h^{(t)}}$ can never exceed 0.25. As we move further back in time, for h_t where $t < T$, the problem becomes evident: each multiplication by $\sigma'(x)$ reduces the gradient's magnitude (Hochreiter and Schmidhuber, 1997, pp. 1740).

For example, the gradient at time step $t - 1$ is:

$$\frac{\partial L}{\partial W_{t-1}} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial W} \quad (2.13)$$

Each gradient multiplication factor is less than 1 (typically 0.25 for sigmoid), which means the gradient diminishes exponentially. After r layers, the gradient generally is less than 0.25^r . Thus, earlier timestamps receive significantly smaller updates than later ones, causing the vanishing gradient problem (Goodfellow et al., 2016, p. 289).

Conversely, attempting to counteract the vanishing gradient problem by using activation functions with larger gradients or initializing weights with larger values can lead to the exploding gradient problem, where the gradients grow exponentially and can become extremely large, leading to unstable updates and divergence during training (Pascanu et al., 2013, p. 2).

To address the vanishing and exploding gradient problems, LSTM networks were developed. LSTMs use a more complex architecture with gating mechanisms that allow the model to maintain long-term dependencies and effectively backpropagate gradients over long sequences, thus mitigating these issues.

2.3 Long Short-Term Memory

Long Short-Term Memory (LSTM) networks are one of the most effective sequence models used in practical applications. Traditional RNNs struggle with retaining information over long sequences because each transition tends to degrade the information from previous states. For example, a network processing text should recognize an opening quotation mark and remember it until a closing quotation mark is encountered. However, RNNs typically fails to retain such information over long texts.

LSTMs addresses this issue by adding gates that control when information is stored, forgotten, and outputted. These gates enable the network to retain relevant information over long sequences and to discard irrelevant information while

also mitigating the exploding/vanishing gradient problem. (Goodfellow et al., 2016, pp. 411).

2.3.1 Architecture

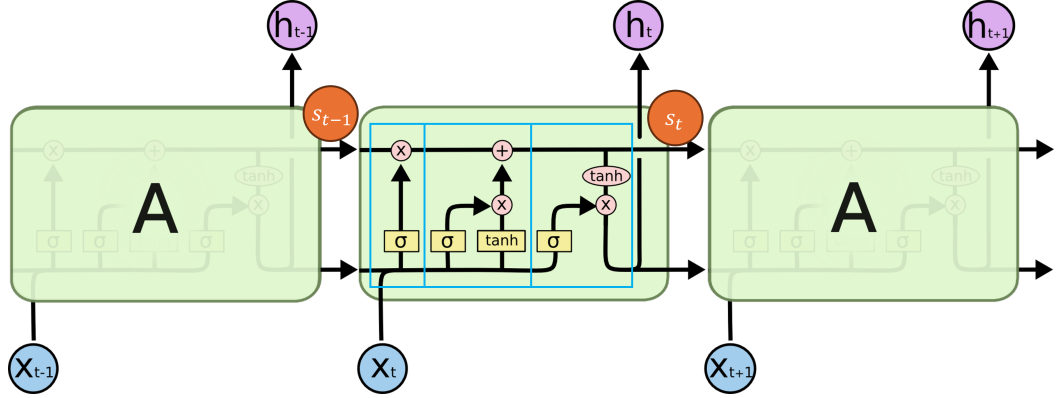


Fig. 2.4: Unrolled LSTM Cells. Each gate is outlined with a blue square: to the left is the forget gate, in the middle is the input gate, and to the right is the output gate. Each cell passes along its cell state (s_t) as well as its hidden state (h_t). Image adjusted from Olah (2015).

An LSTM network is composed of LSTM cells depicted in figure 2.4. The cell contains an internal recurrence (a self-loop), as well as the outer recurrence of a typical RNN. Each cell contains several gates that regulate the flow of information. The primary components of an LSTM cell are (for cell i at timestep t):

- 1) **Cell State** ($s_i^{(t)}$): Represents the long-term memory of the network.
- 2) **Hidden State** ($h_i^{(t)}$): Represents the short-term memory of the network.
- 3) **Forget Gate** ($f_i^{(t)}$): Decides what information should be discarded from the cell state.
- 4) **Input Gate** ($g_i^{(t)}$): Determines what new information should be added to the cell state.
- 5) **Output Gate** ($q_i^{(t)}$): Controls the output of the cell.

Forget Gate

The forget gate f_i is computed as:

$$f_i^{(t)} = \sigma \left(b_i^f + \sum_j U_{i,j}^f x_j^{(t)} + \sum_j W_{i,j}^f h_j^{(t-1)} \right) \quad (2.14)$$

Here, σ is the sigmoid activation function. b_i^f is the bias term, U^f are the input weights, and W^f are the recurrent weights for the forget gate. $x_j^{(t)}$ is the input vector at time t , and $h_j^{(t-1)}$ is the hidden state from the previous time step containing the outputs of all the LSTM cells.

Input Gate

The input gate g_i is computed similarly but with its own parameters:

$$g_i^{(t)} = \sigma \left(b_i^g + \sum_j U_{i,j}^g x_j^{(t)} + \sum_j W_{i,j}^g h_j^{(t-1)} \right) \quad (2.15)$$

The input gate works in conjunction with the forget gate to update the cell state.

Cell State Update

The cell state s_i is updated as follows:

$$s_i^{(t)} = f_i^{(t)} s_i^{(t-1)} + g_i^{(t)} \sigma \left(b_i + \sum_j U_{i,j} x_j^{(t)} + \sum_j W_{i,j} h_j^{(t-1)} \right) \quad (2.16)$$

This equation shows that the cell state is a combination of the previous cell state, modulated by the forget gate, and the new candidate values, modulated by the input gate.

Output Gate

Finally, the output of the LSTM cell $h_i^{(t)}$ is determined by the output gate q_i :

$$q_i^{(t)} = \sigma \left(b_i^o + \sum_j U_{i,j}^o x_j^{(t)} + \sum_j W_{i,j}^o h_j^{(t-1)} \right) \quad (2.17)$$

$$h_i^{(t)} = \tanh(s_i^{(t)}) q_i^{(t)} \quad (2.18)$$

The output gate controls which parts of the cell state are outputted as the hidden state. It does not change the cell state, as the other gates do, but instead judges the importance of the current cell state for the next cells (Goodfellow et al., 2016, pp. 410).

2.3.2 Addressing the Vanishing and Exploding Gradients

Each gate returns a value between 0 and 1. These vectors can be conceptually understood as binary values. Multiplying binary values is comparable to using the $\wedge$ (AND) operator, allowing or disallowing information to flow. To allow gradients to flow through them smoothly, the gates return values between 0 and 1. Each gate works on adjusting the cell state. Its value is changed through multiplication with the forget gate and addition from the input gate. Importantly, it does not use any weights or biases. This lack of weights ensures the network retains important information over long sequences, preventing the exploding/vanishing gradient problem (Hochreiter and Schmidhuber, 1997, pp. 1738).

2.4 Convolutional Neural Network

Convolutional Neural Network (CNN) are a specialized type of neural network primarily used for processing images and other grid-like data structures. They have been highly successful in tasks such as image classification, object detection, and segmentation due to their ability to capture spatial hierarchies in data (Goodfellow et al., 2016, p. 330).

The key idea behind CNNs is their use of convolutional layers that apply filters (also known as kernels) to the input data. These filters slide over the input data,

performing element-wise multiplications and summing the results, which allows the network to detect patterns such as edges, textures, and shapes.

In CNNs, the states in each layer are arranged in a spatial grid structure. This structure preserves the spatial relationships within the data, as each feature value is derived from a small local region of the previous layer. The spatial relationships are crucial for tasks involving images, where the relative positions of pixels carry significant information (Aggarwal, 2021, pp. 255).

Each layer in a CNN is represented as a 3D grid with dimensions corresponding to height, width, and depth. The depth does not refer to the depth of the network itself but instead to the number of channels in the image. For instance, a colored image has three channels: red, green, and blue, each representing an independent property of the first layer. In subsequent layers, the depth corresponds to the number of feature maps.

Let's denote the input to the q -th layer of a CNN as having dimensions $L_q \times B_q \times d_q$, where L_q is the height, B_q is the width, and d_q is the depth. Figure 2.5a depicts an example setup. As we progress through the layers, the depth d_q typically increases, indicating more complex feature maps capturing higher-level representations of the input. While the spatial dimension typically decreases. (Aggarwal, 2021, pp. 256).

Feature maps (or activation maps) are the outputs of the filters applied to the input data. They are analogous to the hidden layers in feedforward networks, representing the learned features at each layer.

CNNs parameters are organized into filters. A filter is typically a small, square matrix that is smaller than the layer to which it is applied. The size of a filter in the q -th layer is defined by $F_q \times F_q \times d_q$, where F_q is the height and width of the filter, and d_q matches the depth of the input (Aggarwal, 2021, p. 258).

2.4.1 Convolution

The convolution operation is fundamental to CNNs, enabling them to detect and learn various patterns in the input data. In a convolution operation, a filter is

applied to every possible position in the image, fully overlapping with the input and performing a dot product. Multiple filters, each with an independent set of parameters, produce multiple feature maps that are stacked together and passed along as the output. The number of filters used in a layer determines the depth of the subsequent layer.

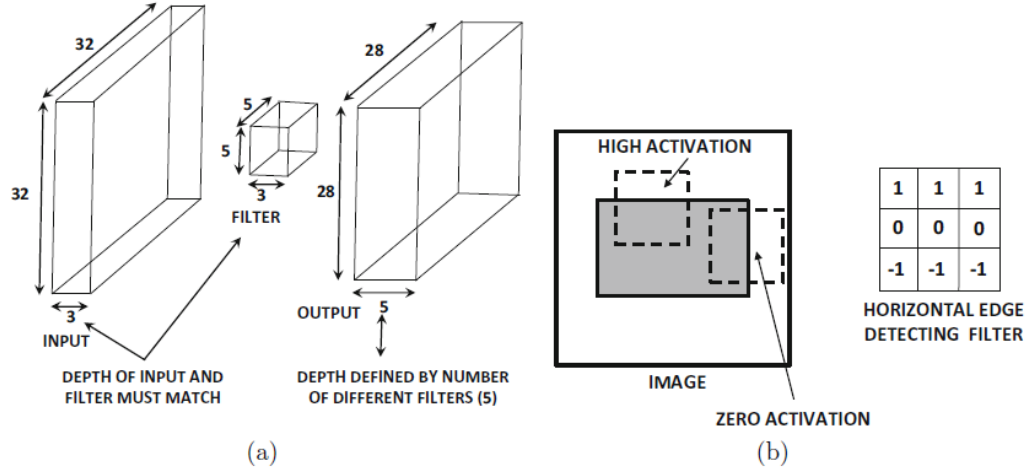


Fig. 2.5: (a) When an input layer of dimensions $32 \times 32 \times 3$ is convolved with a $5 \times 5 \times 3$ filter, the resulting output layer has spatial dimensions of 28×28 . (b) By moving a filter over different sections of the image, we attempt to detect specific features within each area. Image and description as seen in Aggarwal (2021, p. 258).

Intuitively, each filter attempts to identify a specific pattern in a small region of the image. As the depth of the network increases, the detected patterns combine to form more complex shapes. For example, a horizontal edge-detecting filter, like the one shown in figure 2.5 b, will show high activation when a horizontal line is present, zero activation for a vertical line, and intermediate activation for a slanted edge. By sliding the filter across the entire image, key outlines are detected. Combining outputs from multiple filters (e.g., vertical and horizontal edge detectors) allows the network to detect more complex shapes such as squares. As the number of applied filters increases, the complexity of distinguishable shapes also increases.

The size of the filter can be thought of as the visual field. Each spatial location is checked for a specific shape, making the model equivariant to translations. This means that shifting the pixel values will result in a corresponding shift in the

feature values, allowing the network to perceive the presence of particular shapes, irrespective of their specific spatial location.

Let us now formally define the convolution operator. For the p -th filter in the q -th layer, the parameters are defined in the 3D tensor $W^{(p,q)} = [w_{i,j,k}^{(p,q)}]$. The indices i, j , and k define the position in the filter along its height, width, and depth, respectively. The feature maps in the q -th layer are defined by $H^{(q)} = [h_{i,j,k}^{(q)}]$. The input layer, $H^{(1)}$, represents the input image. Convolutions from the q -th layer to the next are defined as:

$$h_{i,j,p}^{(q+1)} = \sum_{r=1}^{F_q} \sum_{s=1}^{F_q} \sum_{k=1}^{d_q} w_{r,s,k}^{(p,q)} h_{i+r-1,j+s-1,k}^{(q)} \quad (2.19)$$

for $i \in \{1, \dots, L_q - F_q + 1\}$, $j \in \{1, \dots, B_q - F_q + 1\}$, and $p \in \{1, \dots, d_{q+1}\}$.

This equation describes the filter w being moved over the values of the q -th layer H . The output has a height of $L_q - F_q + 1$ and a width of $B_q - F_q + 1$. For every filter, the depth d_{q+1} increases (Aggarwal, 2021, pp. 256).

2.4.1.1 Padding

Padding is a technique used in CNNs to maintain the spatial dimensions of the input after the convolution operation, preventing the reduction in size that occurs when filters are applied to the edges of the input. This is known as valid padding. In the absence of such padding, the filter is only applied to positions where it fully fits within the input dimensions, resulting in smaller output feature maps. This approach can lead to the under-representation of border pixels, as they are included in fewer convolution operations, compounding over multiple layers and potentially causing a loss of information.

To address this issue, we use padding to add extra pixels around the border of the input. One common approach is half-padding, which adds $(F_q - 1)/2$ pixels around the border. The padded values are typically set to zero to ensure they do not contribute to the final dot product. This allows the filter to cover the borders sufficiently, maintaining the original spatial dimensions of the input.

Another approach is full-padding, which extends the idea of padding further. In full-padding, the layer is padded such that the filter can be applied even if only one pixel overlaps with the input. This method increases the spatial dimensions of the input significantly by $2(F_q - 1)$ pixels, allowing the entire filter to convolve over the input without reducing the output size. This approach ensures that the border pixels are as well-represented as the central ones, but at the cost of increased computational complexity and potential noise introduction from the padding values (Aggarwal, 2021, pp. 261).

2.4.1.2 Strides

The stride parameter in convolutional operations describes the level of granularity at which the filter is applied across the input data. Previously, we described convolution with a stride of one, meaning the filter moves one pixel at a time for each step. Increasing the stride to two would mean the filter moves two pixels at a time. This effectively reduces the spatial dimensions of the output by approximately half.

Adjusting the stride can be particularly useful in memory-constrained settings, as it reduces the number of computations and the amount of memory required. Moreover, larger strides help to mitigate overfitting by reducing the model's capacity, thus increasing the receptive field. This allows the model to capture more complex features over larger regions of the image. However, this comes at the potential cost of missing finer details within the image, as the filter skips over more pixels with each step (Aggarwal, 2021, pp. 262).

2.4.2 ReLU Activation Function

The *Rectified Linear Unit* (ReLU) is a widely used non-linear activation function in neural networks, including CNN. ReLU is preferred due to its simplicity and effectiveness. For every value in a layer of dimensions, $L_q \times B_q \times d_q$, the ReLU activation function returns thresholded values, effectively setting any value below zero to zero.

$$\text{ReLU}(x) = \max(0, x) \quad (2.20)$$

The effectiveness of ReLU was empirically demonstrated by Krizhevsky et al. (2012) in their work on the AlexNet architecture. They showed that ReLU not only accelerates the convergence of the training process but also improves the overall accuracy of the network. Specifically, ReLU's ability to speed up the training time per epoch translates into faster and more accurate learning.

ReLU also helps mitigate the vanishing gradient problem, which is common in deep neural networks with sigmoid or tanh activation functions. Since ReLU does not saturate in the positive domain, it allows gradients to flow more effectively through the network. (Glorot and Bengio, 2010)

While ReLU has many advantages, it is not without its potential issues. One common problem is the "dying ReLU" phenomenon, where neurons can become inactive and only output zero, effectively dying. This happens when a large number of neurons output zero for all inputs, which can prevent parts of the network from learning. To address this, variations of ReLU, such as Leaky ReLU and Parametric ReLU, have been developed. (Aggarwal, 2021, p. 264)

Leaky ReLU allows a small, non-zero gradient when the unit is not active:

$$\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (2.21)$$

where α is a small constant.

Parametric ReLU is similar to Leaky ReLU, but α is a learned parameter during training (Goodfellow et al., 2016, pp. 193).

2.4.3 Pooling

Pooling layers are an essential component of CNNs, used to reduce the spatial dimensions of the data while preserving its depth. Pooling increases the receptive

field of the subsequent layers, allowing them to detect more complex features from larger portions of the input data (Aggarwal, 2021, pp. 264).

There are different types of pooling operations, the most common being max pooling and average pooling.

Max Pooling

Max pooling involves taking a small region of size $P_q \times P_q$ from the input layer and returning the maximum value within that region. This operation is typically performed with a stride $S_q > 1$ and a pool of size 2×2 , meaning there is no overlap between the pooling regions. This configuration effectively halves the spatial dimensions of the input layer, making it a common choice in many CNN architectures. The resulting layer has the following dimensions:

$$\left[\frac{L_q - P_q}{S_q} + 1 \right] \times \left[\frac{B_q - P_q}{S_q} + 1 \right] \times d_q \quad (2.22)$$

Max pooling returns the most dominant features of the image, such as the brightest pixel, the sharpest edge, or the darkest background, effectively capturing the most salient aspects of the input data. This process helps preserve translation invariance to some extent. Max Pooling also provides some nonlinearity and a greater amount of translation invariance compared to other methods of reducing the spatial footprint, such as a larger stride in the convolution operator (Aggarwal, 2021, pp. 264).

Average Pooling

Average pooling, also known as subsampling, calculates the average value of the selected region $P_q \times P_q$ and returns this average as the pooled value. Average pooling smoothes out features and spreads the feature values to neighboring pixels. Therefore, it does not preserve translation invariance as effectively as max pooling. The resulting layer size is the same as in max pooling, demonstrated in equation 2.22.

Pooling is performed at the level of each activation map, which means the depth of the output layer remains the same as the input layer. However, pooling drastically

reduces the spatial dimensions (height and width) of the input, leading to several benefits.

Reducing the spatial dimensions decreases the number of parameters and computations in the network, therefore decreasing the computational complexity and memory usage. This allows the network to process larger images more efficiently and less prone to overfitting.

2.4.4 Up-Convolutions (Transposed Convolutions)

Up-convolutions, also known as transposed convolutions or deconvolutions, are a type of operation used to increase the spatial dimensions of the input data. They are essentially the opposite of the standard convolution operation. While a regular convolution operation reduces the spatial dimensions of the input, up-convolutions aim to expand these dimensions.

In a standard convolution, a filter (or kernel) of size $F_q \times F_q$ slides over the input image, applying a dot product and reducing each region to a single pixel value. In contrast, up-convolutions reverse this process. Instead of sliding the filter over the input to reduce its size, the filter is moved over each pixel of the input, and the filter values are multiplied by the pixel value to produce a larger output.

Formally, if we denote the input feature map as $H^{(q)}$ with dimensions $L_q \times B_q \times d_q$, and the filter as $W^{(p,q)} = [w_{i,j,k}^{(p,q)}]$ with dimensions $F_q \times F_q \times d_q$, the up-convolution operation can be described as:

$$h_{i,j,p}^{(q+1)} = \sum_{r=1}^{F_q} \sum_{s=1}^{F_q} \sum_{k=1}^{d_q} w_{r,s,k}^{(p,q)} \cdot h_{i-r+1,j-s+1,k}^{(q)} \quad (2.23)$$

for $i \in \{1, \dots, L_{q+1}\}$, $j \in \{1, \dots, B_{q+1}\}$, and $p \in \{1, \dots, d_{q+1}\}$.

Here, L_{q+1} and B_{q+1} are the dimensions of the output feature map, which are larger than the input dimensions.

Up-convolutions offer several benefits, including restoring spatial dimensions and preserving detailed information in the output. It is a key component for tasks such as image segmentation or Generative Adversarial Networks (GANs).

However, they also present challenges. One common issue is the creation of checkerboard artifacts in the output image, which can result from the overlapping of filter applications. This can be mitigated by careful design choices, such as using appropriate initialization schemes and regularization techniques. (Dumoulin and Visin, 2018, pp. 19)

2.5 U-Net

U-Net is a deep convolutional network architecture designed for biomedical image segmentation. Introduced by Ronneberger et al. (2015), the model was used to segment medical images efficiently and was state of the art in 2015, winning multiple image segmentation challenges. It is able to perform precise segmentations with relatively few training images, leveraging extensive data augmentation.

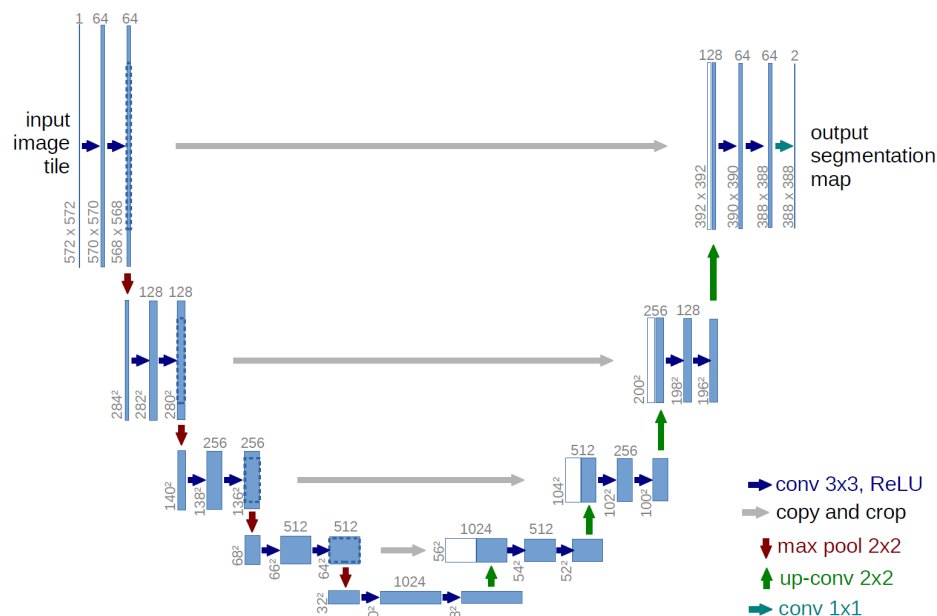


Fig. 2.6: U-Net Architecture as shown in Ronneberger et al. (2015).

The U-Net architecture, illustrated in figure 2.6, is composed of two main parts: a contracting path (also known as the encoder) and an expanding path (also known as the decoder). The contracting path captures context, while the expanding path enables precise localization.

Contracting Path

The contracting path follows the typical architecture of a convolutional network. It consists of repeated applications of two 3×3 convolutions with valid padding, a ReLU activation function, and a 2×2 max pooling operation with a stride of 2.

This series of operations is repeated multiple times, doubling the number of feature channels each time. This allows the network to capture increasingly complex features as the spatial dimensions are reduced. (Ronneberger et al., 2015)

Expanding Path

The expanding path reconstructs the spatial dimensions reduced by the contracting path. It consists of repeated application of an Up-Convolution followed by two 2×2 convolutions that reduce the number of feature channels by half. The output is then concatenated with the feature maps from the corresponding contracting path to provide context information that was captured during the downsampling.

This concatenation is called skip connections and is one of the key innovations of U-Net. These connections concatenate feature maps from the contracting path to the corresponding layer in the expanding path. This technique preserves spatial information that might be lost during downsampling and ensures that the expanding path has access to fine-grained details from the contracting path. (Ronneberger et al., 2015)

With a solid theoretical foundation established, we now turn our attention to the dataset used for training and testing our models. This analysis will set the stage for discussing the implementation details, experimental procedures, and results, leading to our final conclusions.

3 Data

A thorough examination of the data is crucial for understanding the model’s training process and performance. In this chapter, we provide an overview of the dataset, including its origin, structure, and annotations. We also detail the preprocessing steps applied to prepare the data for our experiments. By analyzing the different variations and conditions of the data, we aim to build a deeper understanding of how these factors influence the effectiveness of the proposed model.

3.1 Origin

The dataset includes videos of varying lengths captured from inside the NEB723, a German regional train operating primarily in the northeast of Brandenburg, Germany. These videos were recorded by security cameras mounted on the train’s ceiling, positioned to monitor eight of its doors. The dataset is provided by Inter-automation Deutschland GmbH, the company responsible for the current passenger counting systems used in these trains.

The security cameras on the NEB723 are continuously active, and for this project, specific recordings were extracted, spanning from January to October 2017. Each video captures the door of the train opening, passengers entering and exiting the train and the door closing again. The accompanying labels indicate the number of passengers who have entered and exited the train for each sequence. Additionally, the labels include metadata specifying the exact time the video was recorded and the specific door at which it was captured.

Due to the camera’s overhead angle, facial features of passengers are typically obscured, enhancing privacy. Despite this, the data must be handled with utmost care to ensure the privacy and anonymity of individuals. A critical goal of this

project is to develop a passenger counting system that does not rely on personal data, such as facial recognition. The system is designed to function as a "black box," where no personally identifiable information can be extracted or inferred.

Passengers are informed about the surveillance through visible stickers and signs inside the train, ensuring transparency and compliance with privacy regulations.

3.2 Annotations

Along with the 2D videos used for training and testing, the camera positioned above the train entrance also recorded depth videos. Human annotators used these depth videos to create accurate labels for the dataset. Notably, the depth videos themselves were not used in the experiments presented in this thesis.

In Germany, passengers below the age of 6 do not require a ticket and, therefore, do not count towards the public transport revenue. To distinguish children from other passengers, a height threshold of 130 cm was used. Individuals below this height were classified as children under 6 and were excluded from the passenger count.

This presents a significant challenge for our model, as it must learn to differentiate between passengers who should be counted and those who should not. The ability to correctly identify and exclude children from the count is critical for ensuring the accuracy of our predictions.

The dataset is annotated with labels provided in CSV format. Each entry in the CSV file corresponds to a video, with matching filenames between the videos and labels. The annotations include the number of people entering and exiting the vehicle, as well as the specific door, time, and day of the event.

It is important to note that the labels only provide the total count of passengers after the door sequence is completed rather than tracking the exact timestamp at which a passenger enters or exits the vehicle.

3.3 Collection Conditions

The videos were recorded using the Hella Aglaia Stereo Camera APC-R. These cameras are positioned above eight doors of the train, providing a clear overhead view of the door area and its immediate surroundings. The videos are captured as black and white videos and stored in HDF5 format, which ensures efficient storage and retrieval (Land Transport Guru, 2018).

The dataset consists of video recordings taken at various train stations throughout Brandenburg, covering different times of the day and capturing a wide variety of conditions. This includes different lighting levels (e.g., early morning, midday, evening), diverse weather conditions (e.g., sunny, rainy, overcast), and varying crowd densities.

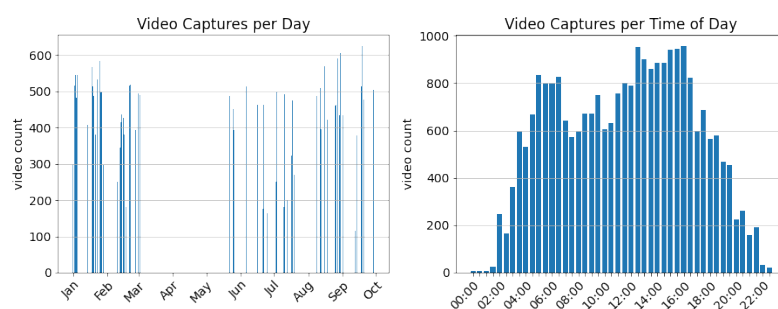


Fig. 3.1: The left graph shows the frequency of video recordings across different days of the month. The right graph shows the time distribution of video recordings throughout the day..

Figure 3.1 illustrates the distribution of video data collected throughout the experiment period. The left-hand graph shows the frequency of video captures by day, highlighting that the majority of recordings were done during winter and summer, while fewer videos were captured during the spring months.

The right-hand graph displays the distribution of video recordings across different times of the day. The majority of videos were recorded during typical operating hours, with 72% of recordings taking place between 6 AM and 6 PM. This concentration aligns with peak passenger travel times, ensuring the dataset is representative of the busiest periods when accurate passenger counting is most critical. The dataset also includes recordings outside peak hours.

3.4 Distributions

The dataset comprises 25,767 videos, with a combined total length of approximately 145.7 hours.

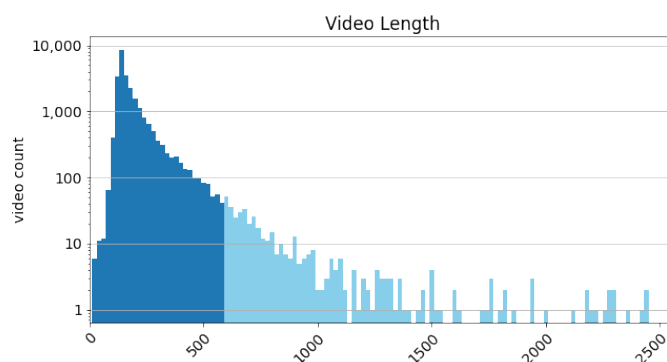


Fig. 3.2: Distribution of video lengths. The plot uses a logarithmic y-axis, showing the number of videos per bin. The x-axis represents the number of frames per video, grouped in bins of 20 frames each. Videos longer than 600 frames are presented using a lighter shade.

Figure 3.2 illustrates the distribution of video lengths. The videos vary significantly in duration, with 33.5% (8,640 videos) being between 120 and 140 frames long. The average video length is 203 frames, with the shortest being 34 frames and the longest being 2,445 frames. Recorded at 10 frames per second (fps), the longest video is approximately four minutes in duration, while the shortest video lasts about three seconds. Most videos capture the sequence of the door opening, passengers entering or exiting, and the door closing. However, some videos are truncated, missing either the opening or closing event, often due to passengers manually interfering with the doors. Approximately 98.3% of the videos are shorter than 1 minute, which serves as the upper limit for the videos used in our experiments. Videos longer than that often include some type of disruption.

Figure 3.3 displays the distribution of videos according to the number of passengers entering or exiting the train. The y-axis uses a logarithmic scale to illustrate the range of values better, while the x-axis is linear, showing the number of passengers. On average, 2.12 people enter, and 2.14 people exit per video. Interestingly, 34.7% of videos show no passenger entering, and 48.8% of videos show no passen-

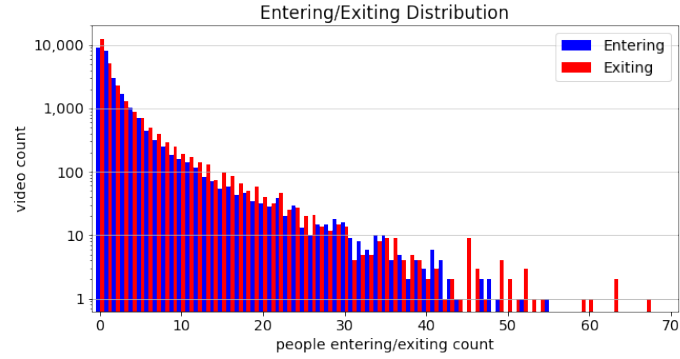


Fig. 3.3: Distribution of videos based on the number of people entering or exiting. The y-axis uses a logarithmic scale to represent the number of videos, while the x-axis indicates the number of people entering or exiting. Blue bars represent people entering, and red bars represent people exiting the train.

ger exiting. Moreover, 91% of videos depict five or fewer people entering, and 89% depict five or fewer people exiting.

The data distribution is heavily skewed towards videos with fewer passengers, which indicates a class imbalance in the dataset. This imbalance could potentially bias our models towards undercounting the count of passengers.

3.5 Preprocessing

The data used in this study has been preprocessed into three distinct datasets, each differing in the cutout and resolution of the original surveillance footage. These variations allow us to evaluate the model’s performance under different resolutions and framing conditions.

The top row of Figure 3.4 shows Dataset A, which has a 80×106 pixels resolution. This resolution represents a scaled-down version of the original footage to ensure a manageable model size and reasonable training time. This dataset captures the most detailed view of the entry area, including the door and a few meters into the entryway. Passengers are visible in this area, often standing and waiting for the door to open.

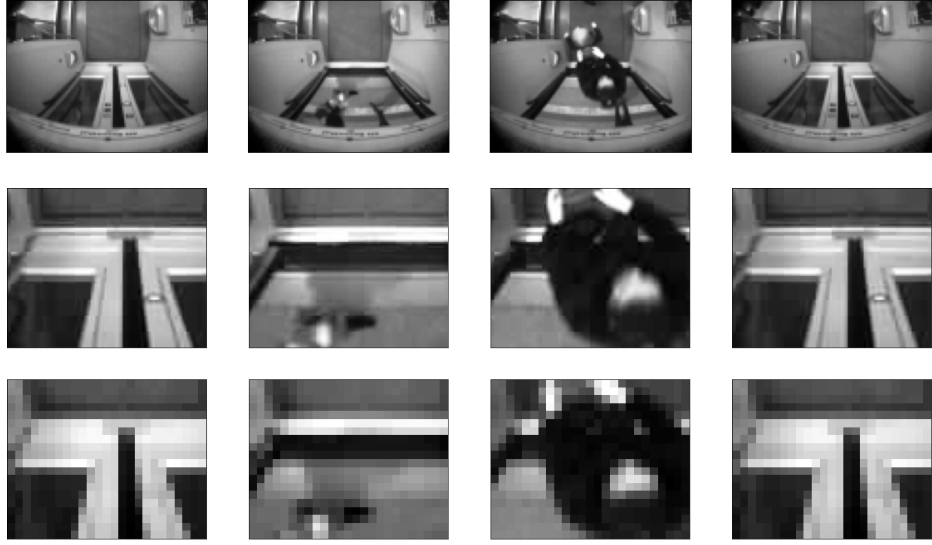


Fig. 3.4: Example frames from each dataset. The top row shows Dataset A, the middle row Dataset B, and the bottom row Dataset C. Depicted are the same frames of the same video in the different datasets.

The middle row depicts Dataset B, which maintains approximately the same resolution of 80×100 pixels as Dataset A but focuses more narrowly on the area immediately in front of the door. By reducing the cutout size, this dataset captures less extraneous information, focusing on the core region where passengers enter or exit the train.

The bottom row depicts Dataset C, which shares the same framing as Dataset B but at a reduced resolution of 20×25 pixels. This lower resolution reduces computational load and is intended to decrease training time, though it may sacrifice some level of detail.

These datasets enable model performance evaluation under different resolutions and framing conditions. By comparing these datasets, we aim to determine whether reducing overhead information and focusing on the essential area can enhance model robustness.

To further optimize computational efficiency, the number of frames per video was reduced by only sampling every second frame, effectively halving the frame rate.

This reduction aims to decrease computational complexity while retaining essential information for passenger counting.

3.6 Split

To ensure a robust evaluation of the model, the dataset is divided into training, testing, and validation sets. The videos are clustered by date, and the 60 days of data are split into 60%, 20%, and 20% for training, testing, and validation, respectively. This method of temporal splitting maintains the chronological coherence of the data, ensuring that all videos recorded on the same day remain within the same subset.

This approach provides comprehensive coverage of different lighting conditions, weather variations, and other environmental factors across each dataset, which helps enhance the generalizability of the model.

Moreover, this temporal splitting strategy prevents data leakage, where information from the test or validation sets could inadvertently influence the training process. Keeping each set distinct in terms of time periods ensures that evaluation metrics accurately reflect the model's ability to generalize to completely unseen data.

3.7 Augmentation

To mitigate overfitting and enhance the generalizability of the model, the data is randomly augmented before each training epoch. Data augmentation introduces variations in the training data, which supports the model learning to handle different scenarios and prevents it from memorizing specific features.

Figure 3.4 shows that the door is always positioned towards the bottom of each frame. We apply horizontal flipping to the videos to create diversity in the dataset while preserving spatial consistency by keeping the door at the bottom of the frame.

In addition to horizontal flipping, we also apply temporal flipping, where the sequence of frames is reversed. This augmentation essentially makes passengers appear to walk backward through the door, reversing the flow of people entering and exiting. Videos that initially show passengers entering now depict them leaving, and vice versa. Temporal flipping requires modifying the video sequences and adjusting the associated labels accordingly to maintain consistency.

The selection of augmentation techniques is randomized during each training epoch, meaning no two epochs use the same set of augmented data. This randomness provides a richer training experience, allowing the model to generalize effectively across different situations and preventing it from becoming overly dependent on specific patterns present in the original dataset.

4 Methods

With a solid understanding of the theory behind each part of U-Net and LSTM and an understanding of the dataset, we can now introduce the combined model and systems surrounding it.

This paper aims to discuss the combined use of U-Net and LSTM to count passengers in PUTs. To achieve this, a series of experiments were performed to show the effect of different surveillance cutouts and various amounts of encoder and decoder blocks in the U-Net. These results will be compared against a model using only LSTM to finally decide if the U-Net has improved the quality of the predictions.

Models using Normalisation after each convolution and a simple CNN+LSTM Setup are also tested.

To test these we run our experiments on a *High Performance Cluster* (HPC) provided by the Technical University of Berlin. These clusters allow for multiple experiments to be run simultaneously.

The upcoming chapters will discuss the implementations of the model on the HPC.

4.1 Implementation

4.1.1 U-Net

The U-Net architecture for this project is implemented using TensorFlow 2.6.2, leveraging the high-level Keras API for building and training the neural network

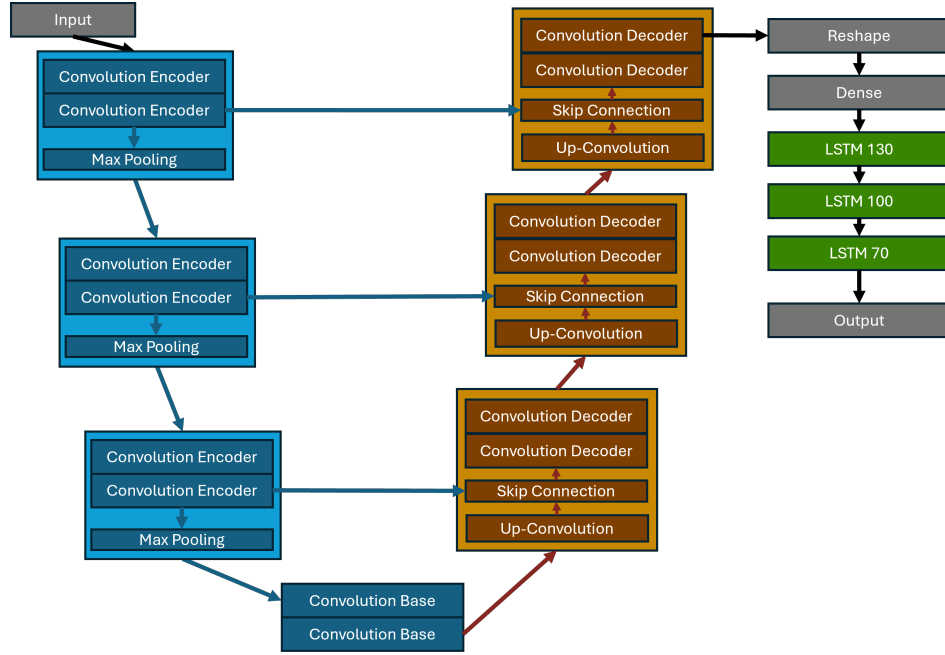


Fig. 4.1: Example architecture showing both U-Net and LSTM and how they are connected to each other. With three encoder and three decoder blocks.

models. The implementation closely follows the architectural principles described in Section 2.5 and can be visually understood in the left part of figure 4.1.

The input to the U-Net is a sequence of video frames with dimensions (batch size, time step, column, row, value). The U-Net processes these inputs to extract spatial features, which are then reshaped and passed to the LSTM layers.

The U-Net consists of multiple encoder and decoder blocks.

An encoder block is implemented using two consecutive `Conv3D` Layers followed by a `MaxPooling3D` Layer. The model uses `Conv3D` layers instead of `Conv2D` to capture temporal information across consecutive frames.

A decoder block is implemented using a `Conv3DTranspose` layer for up-convolutions. Its output might be cropped by 1 pixel using `Cropping3D` because of rounding errors, then we concatenate both the `Conv3DTranspose` output with the output of the equivalent encoder block using the `Concatenate` layer to create the skip connections. This is then passed on to 2 consecutive `Conv3D` layers.

For models with an asymmetrical encoder-decoder configuration, the skip connections are created by linking the output of each decoder block with the corresponding encoder block that is closest to it in depth. Specifically, the first decoder block receives input from the network's base and connects with the nearest encoder block, creating the initial skip connection. Subsequent decoder blocks continue this pattern, each receiving the output from the previous decoder block along with input from the next encoder block further from the base. This design ensures that the decoder has progressively higher-resolution feature maps to work with, improving spatial detail reconstruction.

Every Convolution layer uses a consistent kernel size of $(1, 2, 2)$ throughout the network. Each Max Pooling operation uses the same kernel to preserve temporal information but reduces only the spatial dimensions.

The number of filters used in the convolutions doubles per encoder block, starting with four filters. This exponential increase allows the network to capture more complex features at deeper levels. The number of filter halves for each decoder block mirroring the encoder structure.

ReLU activation is used after each convolutional layer to introduce non-linearity.

After the U-Net extracts spatial features from the input frames, these features are fed into the LSTM layers to model temporal dependencies.

4.1.2 LSTM

The architecture consists of several layers, each serving a specific purpose. First, the Reshape Layer transforms the 4D output from the U-Net into a 3D tensor suitable for input to the LSTM layers, flattening the spatial dimensions and feature channels into a single dimension. A Dense Layer is then introduced to reduce the input dimensionality to the LSTM, facilitating the learning of a compact representation of the features. An Activation Function using Leaky ReLU is applied after the dense layer to introduce non-linearity.

Subsequently, there are three LSTM Layers with 130, 100, and 70 units, respectively, selected based on their efficiency as demonstrated by empirical results and their

use in related work on APCs (Jahn, 2019)(Shirazi, 2021). Finally, the Output Layer consists of a dense layer to generate the output predictions, followed by a Leaky ReLU activation to ensure non-negative outputs.

4.2 Cost Function

The cost function is a crucial component in the training of the combined U-Net and LSTM architecture. The cost function used in this project is custom-designed to address not only the primary prediction error but also other constraints that are important for ensuring a robust and stable model. This section details the components of the cost function and the motivation behind its design.

4.2.1 Primary Loss

In an ideal scenario, each video would be labeled with precise timestamps indicating when a person enters or exits the train. However, the dataset available for this study contains only cumulative counts for the number of people entering and exiting at the end of each video. This poses a significant challenge. If the model only receives feedback on the error at the final frame of each video, the model tends to assume that no individuals are entering or exiting throughout most of the sequence, resulting in poor training performance.(Jahn, 2019)

To overcome this limitation, we employed video concatenation during training. Specifically, five videos are randomly selected and concatenated into a single longer video sequence. This concatenation strategy increases data granularity by providing incremental counts at the end of each individual video within the concatenated sequence instead of a single cumulative count at the end. This method introduces a series of smaller increments, forming a step-wise progression and enabling the model to learn more effectively. Figure 4.2 illustrates this concept.

The primary objective of the model is to keep its predictions within these upper and lower bounds, thereby ensuring that predictions remain within a realistic range throughout the inference process.

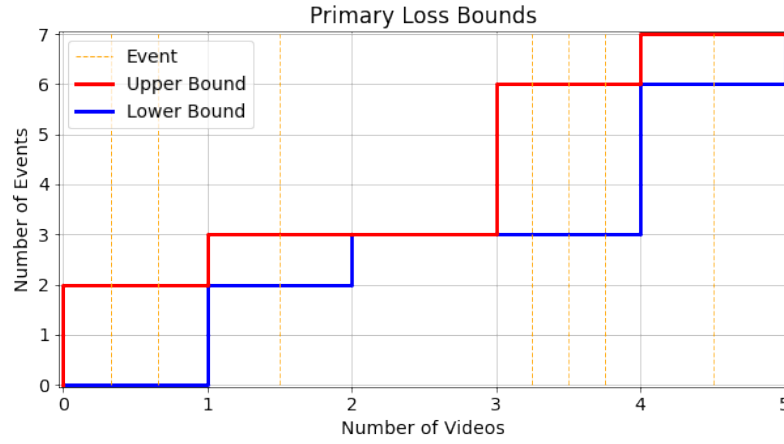


Fig. 4.2: Visualization of bounds used for the primary loss..

4.2.2 Auxiliary Losses

In addition to the primary loss, auxiliary losses are applied to introduce additional constraints on the model's behavior. These auxiliary losses help to improve prediction quality by enforcing stability, smoothness, and alignment with desired characteristics. The auxiliary losses include:

Integer Approximation Error

This loss term encourages the model's predictions to be close to integer values, which is important since the goal is to count passengers. The error is calculated as the distance between the predicted value and its nearest integer.

Temporal Stability

Temporal stability penalizes significant fluctuations between consecutive frame predictions, ensuring that the model's output remains smooth over time. The model should not produce drastic changes in passenger count between frames unless there is a valid reason. It is a critical component for video-based processing.

Freezing Prediction for Padding

The freezing loss is designed to ensure prediction stability during sequences with less activity. During such periods, there is little new information for the model

to process, which could result in fluctuating predictions. The freezing loss penalizes changes in the model's predictions during these moments by encouraging the model to maintain its previous output, effectively "freezing" the prediction when relevant activity is reduced. This helps ensure consistent and stable predictions, especially during less dynamic periods, such as door opening or closing sequences.

The auxiliary losses are combined and averaged to produce a final auxiliary loss value, which is then scaled and added to the primary loss to form the overall cost function.

4.3 Reset On Stall

During the early stages of development, a recurring issue emerged: many experiments failed prematurely during training, regardless of variations in U-Net depth or dataset. To address this issue, a Reset on Stall system was implemented to enhance model training stability and performance.

The system monitors the model's performance throughout its training process. The model's validation accuracy is recorded every five epochs. If there is an improvement, its weights are saved. The key goal is to detect and mitigate instances where the model is stuck in a local minimum, preventing further progress. The system employs both soft and hard resets based on the model's validation accuracy and improvement over epochs.

A soft reset is triggered when the model's validation accuracy falls below a threshold of 70% and shows no improvement for 100 consecutive epochs. When these conditions are met, the system reverts the model to its state from 100 epochs prior, including resetting the epoch count and restoring the weights to their previous values. This process often helps the model escape local minima, allowing it to make progress upon retraining. The model is allowed up to three soft resets around the same epoch before a hard reset is initiated.

A hard reset is performed if the model undergoes three soft resets within 20 epochs of each other. The model is completely reset and retrained from the beginning

during a hard reset. This approach ensures that the model does not remain stuck in a suboptimal state and has another opportunity to learn from scratch.

Each model is allowed to train for a maximum of 30 hours or up to 1500 epochs, whichever comes first. If a model cannot complete the full 1500 epochs due to multiple soft or hard resets, the best-performing model up to that point is selected as the final result.

Models that undergo hard resets often do not reach the total epoch limit. They are not considered for analysis if they do not reach 250 epochs. A list of failed experiments, that are not included in the analysis, can be found in the appendix B.

This system significantly enhances the training stability and performance of all models.

4.4 Performance Measures

We utilized several metrics to evaluate our models' performance effectively: Accuracy, Balanced Accuracy, and *Relative Global Bias*. Each metric is calculated separately for entering and exiting passenger counts. An average of the two is presented. These measures provide a comprehensive understanding of the model's performance under various conditions.

4.4.1 Accuracy

Accuracy is one of the most straightforward performance measures and is defined as the ratio of correctly predicted instances to the total instances in the dataset. In our context, a prediction is considered correct if the model predicts the exact number of passengers that entered or exited, matching the actual count. It is given by the formula:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \times 100 \quad (4.1)$$

In this study, accuracy and all other performance measures are calculated in three steps: first, the accuracy for all entering passengers is computed, then the accuracy for all exiting passengers is determined, and finally, both accuracies are averaged. This stepwise approach ensures that both entry and exit predictions are evaluated independently before deriving an overall accuracy score.

While accuracy is useful, it can be misleading in cases where the dataset is imbalanced. For instance, if the number of passengers entering and exiting the train is skewed, a model might predict the majority class more frequently, resulting in a high accuracy but poor performance in predicting the minority class (Brodersen et al., 2010).

4.4.2 Balanced Accuracy

Balanced Accuracy is employed to address the class imbalance in our dataset, ensuring a fair evaluation of the model's performance across different passenger counts. This metric calculates the accuracy for each group and then averages them, giving equal weight to each group.

In our system, Balanced Accuracy is calculated as follows:

$$A(e) = \frac{\text{Correct predictions in class } e}{\text{Total number of predictions in class } e} \quad (4.2)$$

$$A(\neg e) = \frac{\text{Correct predictions outside class } e}{\text{Total number of predictions outside class } e} \quad (4.3)$$

$$\text{Balanced Accuracy for } e = \frac{A(e) + A(\neg e)}{2} \times 100 \quad (4.4)$$

First, we identify the unique groups for entering passengers, E . Each group $e \in E$ includes all videos with the same number of people entering. For each group e , we calculate the accuracy within the group $A(e)$ and the accuracy for all other instances combined $A(\neg e)$. The balanced accuracy for class e is then the average of these two accuracies. The model balanced accuracy is the average of the balanced accuracy for every class. (Brodersen et al., 2010).

4.4.3 Relative Global Bias

Relative Global Bias (RGB) is a specific measurement used in public transportation to evaluate the passenger load on each mode of transportation. This metric is crucial for accurately distributing ticket sales among the various transportation subcompanies. It focuses on the total sum of missed predictions, allowing the model to compensate for undercounting and overcounting symmetrically.

The RGB measure is defined as follows:

$$\text{RGB} = \frac{\sum \hat{y} - \sum y}{\sum y} \times 100 \quad (4.5)$$

Here, $\sum \hat{y}$ represents the total predicted passenger count, and $\sum y$ represents the actual passenger count.

This measure is particularly important for ensuring that overcounts and undercounts balance out, providing a more accurate and fair distribution of passenger counts across different transportation services. By focusing on overall counts rather than individual predictions, RGB offers a holistic view of the model's performance in capturing the true passenger load dynamics.

4.5 Batch Size

Our models are trained with a batch size of 32. Each batch contains 32 sets of concatenated videos. Since each set includes five videos, the total number of individual videos processed per batch is 160.

5 Experiments

This chapter presents a series of experiments to evaluate the performance of the proposed U-Net and LSTM-based model for passenger counting. The experiments are organized in a progressive manner, including a control experiment and various model designs. The following experiments were conducted:

- 1) **Control Experiment - LSTM Only:** This experiment establishes a baseline by evaluating the performance of the LSTM model without convolutional preprocessing layers.
- 2) **U-Net with Varying Encoder/Decoder Blocks:** We explore how different configurations of encoder and decoder blocks affect the model's performance.
- 3) **U-Net with Batch Normalization:** To address issues of instability and convergence, we add batch normalization layers to the U-Net architecture and analyze its impact on the performance.
- 4) **Simple CNN and LSTM Combination:** Lastly, we replace the U-Net with a simpler CNN to assess whether the added complexity of the U-Net is necessary for achieving a strong performance.

For every experiment, the number of frames is reduced by removing every second frame in order to lower computational complexity.

The LSTM component stays consistent throughout all experiments, with 130 LSTM units in the first layer, 100 in the second layer, and 70 units in the final layer. By keeping the LSTM architecture consistent, any performance differences can be attributed to the inclusion of U-Net preprocessing.

Each configuration was tested ten times to ensure the consistency of results. Experiments that did not reach at least 250 epochs were excluded from the analysis due to insufficient training time. Failures were primarily attributed to insufficient server memory or excessive restarts caused by the reset-on-stall mechanism. In most cases, only a few runs failed; however, in instances where frequent failures occurred, additional details are provided in the corresponding sections. Appendix B shows a complete list of failed runs.

Model Performances are presented in a series of boxplot graphs. For a numeral presentation, all performance measures can be found in numbers in the appendix A.

5.1 Control Experiment - LSTM Only

To assess the impact of the U-Net preprocessing on model performance, we conducted a control experiment using only the LSTM component without the U-Net. This experiment serves as a baseline to determine whether the inclusion of the U-Net improves the accuracy and reliability of the system compared to using the LSTM by itself.

The LSTM-only model processes raw video frames directly, without any convolutional preprocessing to extract spatial features. To handle high-dimensional input, a reshape layer and a dense layer are incorporated at the beginning of the model, mirroring the structure in the combined U-Net and LSTM architecture, which can be seen in figure 4.1.

Figure 5.1 visualizes the average accuracy, balanced accuracy, and relative global bias for the LSTM-only experiments across three datasets, A, B, and C.

Boxplots are used to provide an intuitive visualization of the model’s performance metrics, depicting locality, spread, and skewness. Each boxplot shows key components. The median value of the dataset is represented by a red line within each box, providing a central tendency measure of the data. The mean value of the data is indicated by a green line, which is useful for understanding the average performance of the models. The left and right edges of the box represent the first quartile

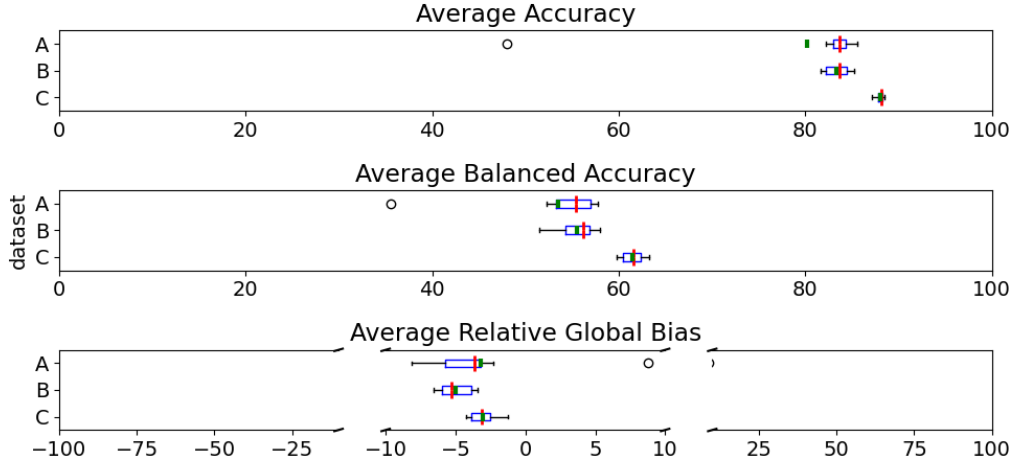


Fig. 5.1: Average Performance for LSTM Experiments. Red lines indicate median values, green lines indicate mean values.

(Q1) and third quartile (Q3), which correspond to the 25th and 75th percentiles of the data, respectively. The interquartile range (IQR) is the difference between Q3 and Q1, capturing the middle 50% of the data. Whiskers extend from the box to the last data point within the range defined by $Q1 - 1.5 \times IQR$ on the left and $Q3 + 1.5 \times IQR$ on the right, providing insight into the spread of the data without being overly influenced by extreme values. Any data points that fall beyond the whiskers' range are displayed as circles, representing outliers that may indicate unusually high or low-performance values, potentially highlighting instability in the model's training. (Dutoit, 2012, p. 29)

The boxplot for "Average Relative Global Bias" features a split x-axis to allow detailed examination of values close to zero while also including the full range for comparison. The x-axis consists of three segments: the first segment spans values from -100 to -10 (representing consistently undercounting results), the second segment zooms in on the range from -10 to 10, in which most models perform closely to the target value, and the third segment extends from 10 to 100 (representing consistently overcounting results). Each of these sections is presented with equal length, effectively zooming into the central range.

The LSTM-only model was evaluated on three datasets: A, B, and C. On average, the models achieved an accuracy of approximately 84%, with Dataset A showing an average accuracy of 80.25% and a relatively high variance of 11.37%. Dataset B yielded an average accuracy of $83.39\% \pm 1.3$, while Dataset C achieved the highest average accuracy of $88.07\% \pm 0.41$.

The improvement in both accuracy and stability with Dataset C can be attributed to its reduced input complexity.

This trend was consistent with the average balanced accuracy, which, as expected, was lower across all datasets due to the imbalanced nature of the data. The average balanced accuracy across all three datasets was 56.83%.

The relative global bias was notably high compared to other experiments presented later in this thesis. Specifically, the average relative global bias was -3.77%, indicating a consistent undercounting of passengers in all model configurations.

Insights

The LSTM-only model showed high variance in accuracy, indicating instability without U-Net preprocessing. The high relative global bias suggests that the LSTM-only model tends to undercount passengers. With its lower resolution and focus on the door region, Dataset C outperformed the higher-resolution datasets A and B, highlighting that simpler input features can lead to better performance in a simple model setup.

5.2 U-Net with Varying Encoder/Decoder Blocks

In this section, we investigate whether a varying number of encoder and decoder blocks in the U-Net impacts the overall model performance.

5.2.1 3 Encoder Blocks

In this experiment, we employ three encoder blocks while varying the number of decoder blocks from 0 to 3.

5.2.1.1 Dataset A

For Dataset A, the output at the base of the U-Net returns frames with a resolution of 10×14 with 32 filters.

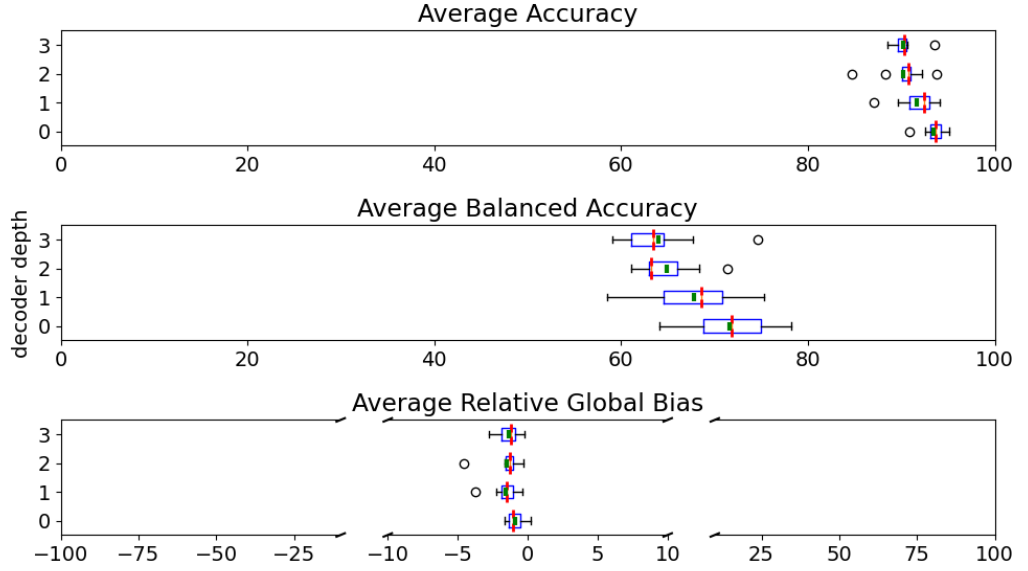


Fig. 5.2: Average Performance in 3 Encoder Blocks Experiment using Dataset A. Three encoder blocks and a varying number of decoder blocks from 0 to 3 were used. Red lines indicate median values, green lines indicate mean values.

Figure 5.2 illustrates our results, which indicate that decreasing the number of decoder blocks generally improves accuracy, with the best-performing model using zero decoder blocks achieving an accuracy of $93.46\% \pm 1.29\%$. Balanced accuracy remained lower, with the best average performance being $71.59 \pm 4.87\%$. Relative global bias was consistently low, with the model using zero decoder blocks achieving the best result at $-0.92\% \pm 0.64\%$.

5.2.1.2 Dataset B

Dataset B followed the same experimental setup as Dataset A. The primary difference lies in the cutout used, as described in Chapter 3.5.

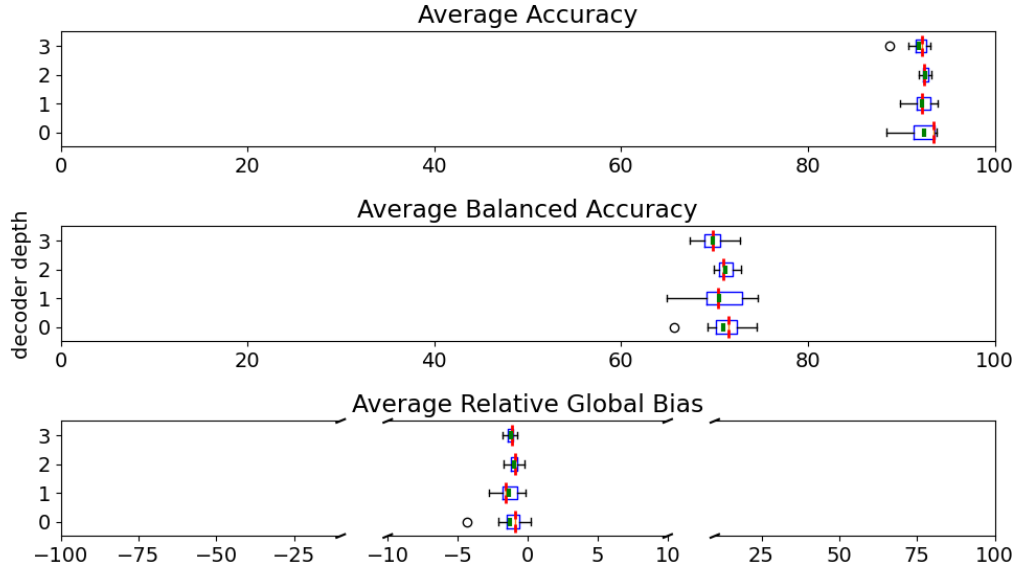


Fig. 5.3: Average Performance in 3 Encoder Blocks Experiment using Dataset B. Three encoder blocks and a varying number of decoder blocks from 0 to 3 were used. Red lines indicate median values, green lines indicate mean values.

As shown in Figure 5.3, decoder depth did not seem to make a big difference for Dataset B compared to Dataset A. In general all configurations performed similarly well. The model with two decoder blocks achieved the highest average accuracy at $92.56\% \pm 0.43\%$, slightly outperforming the model with three decoder blocks ($91.58\% \pm 1.29\%$).

All models using Dataset B outperformed Dataset A in terms of balanced accuracy, with the model employing two decoder blocks achieving an average balanced accuracy of $71.19\% \pm 1.02\%$.

The relative global bias remains consistently close to zero, with the model using two decoder blocks showing a small bias of $-0.97\% \pm 0.46\%$.

5.2.1.3 Dataset C

Dataset C has a significantly smaller resolution compared to the other datasets, with each frame sized at 20×25 pixels. Therefore, the frame size is four times smaller than in the other datasets. At its base, the U-Net returns a frame size of 3×4 with 32 filters.

Due to the reduced frame size, and therefore decreased model complexity, every frame is used without skipping.

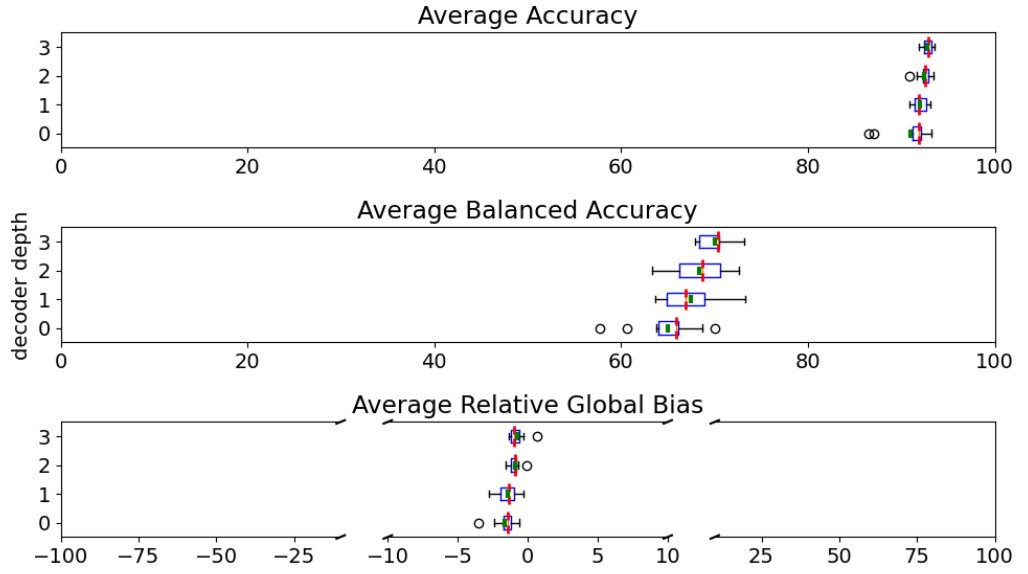


Fig. 5.4: Average Performance in 3 Encoder Blocks Experiment using Dataset C. Three encoder blocks and a varying number of decoder blocks from 0 to 3 were used. Red lines indicate median values, green lines indicate mean values.

Figure 5.4 illustrates that the model with three decoder blocks showed the best model performances with an average accuracy of $92.78\% \pm 0.57\%$, an average balanced accuracy of $70.03\% \pm 1.82\%$ and a relative global bias of $-0.77\% \pm 0.61\%$.

In contrast to Dataset A, using Dataset C, the performance of the model slightly increased with the number of decoder blocks.

This model returned the second-best accuracy score out of all 3-encoder block experiments, second only to Dataset A with 0 decoder blocks.

Insights

The models consistently performed slightly better for passengers boarding compared to passengers alighting the train across all configurations. Dataset B exhibited greater stability than Dataset A, suggesting that a smaller cutout may enhance the model. Dataset C, with lower resolution frames, produced reliable results using the original U-Net design with three encoder and three decoder blocks, indicating that reduced spatial resolution can improve performance in a symmetrical U-Net configuration. Furthermore, Datasets A and B revealed that non-symmetrical models can be just as effective, achieving their best performance in configurations with zero decoder blocks.

5.2.2 5 Encoder Blocks

In the previous experiment, we observed that Dataset C, with significantly smaller frame dimensions of 20×25 pixels compared to Dataset A's 80×106 pixels, performed well, despite reduced spatial information. The U-Net in Dataset C reduced the frame size to 3×4 pixels with 32 features at its base. This led us to hypothesize that increasing the depth of the encoder in the U-Net for Datasets A and B could enhance model performance by capturing more abstract representations while reducing spatial dimensions to a level similar to Dataset C.

To test this hypothesis, we increased the number of encoder blocks from three to five for Datasets A and B, effectively reducing the frame size at the U-Net base to 3×4 pixels and increasing the number of feature channels to 128.

In this experiment, we varied the number of decoder blocks from 0 to 5.

5.2.2.1 Dataset A

Figure 5.5 presents the average performance metrics for Dataset A with five encoder blocks.

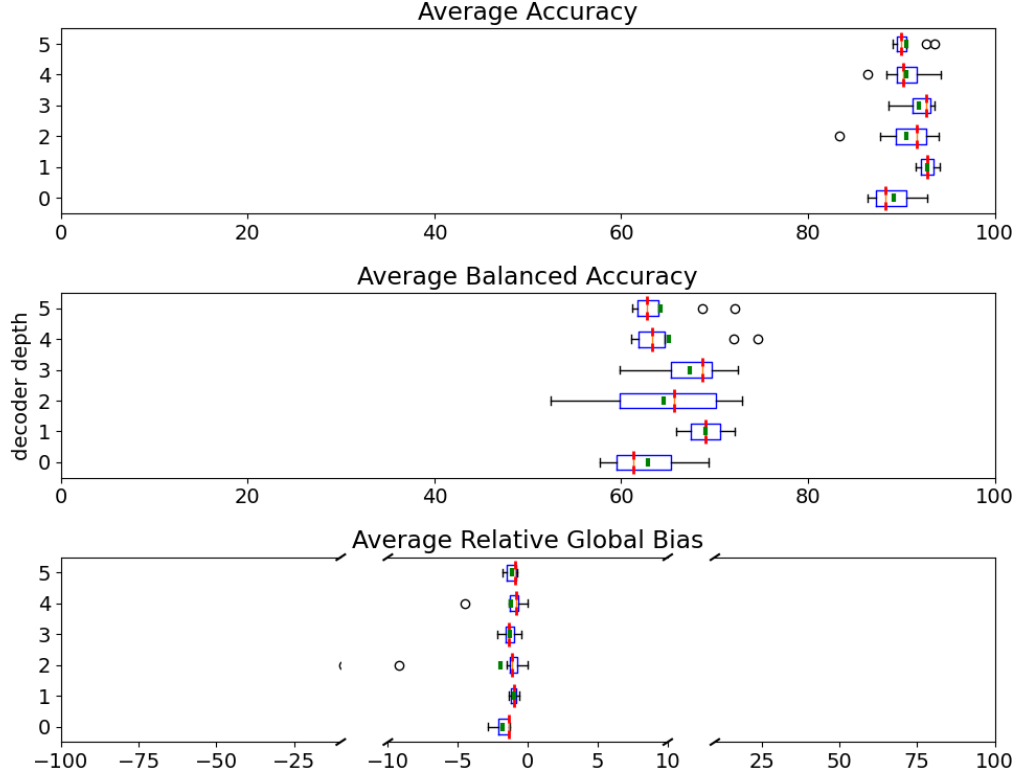


Fig. 5.5: Average Performance in 5 Encoder Blocks Experiment using Dataset A. Five encoder blocks and a varying number of decoder blocks from 0 to 5 were used. Red lines indicate median values, green lines indicate mean values.

The model faced significant convergence challenges, particularly with configurations using 0 and 1 decoder blocks, where only 2 and 3 models, respectively, managed to train for more than 250 epochs.

Among the other configurations, the model with three decoder blocks achieved the highest average accuracy at $91.86\% \pm 1.91\%$, a balanced accuracy of $67.29\% \pm 4.56\%$, and a relative global bias of $-1.29\% \pm 0.53\%$. As the number of decoder blocks increased beyond three, performance declined.

5.2.2.2 Dataset B

Dataset B followed the same experimental procedure as Dataset A.

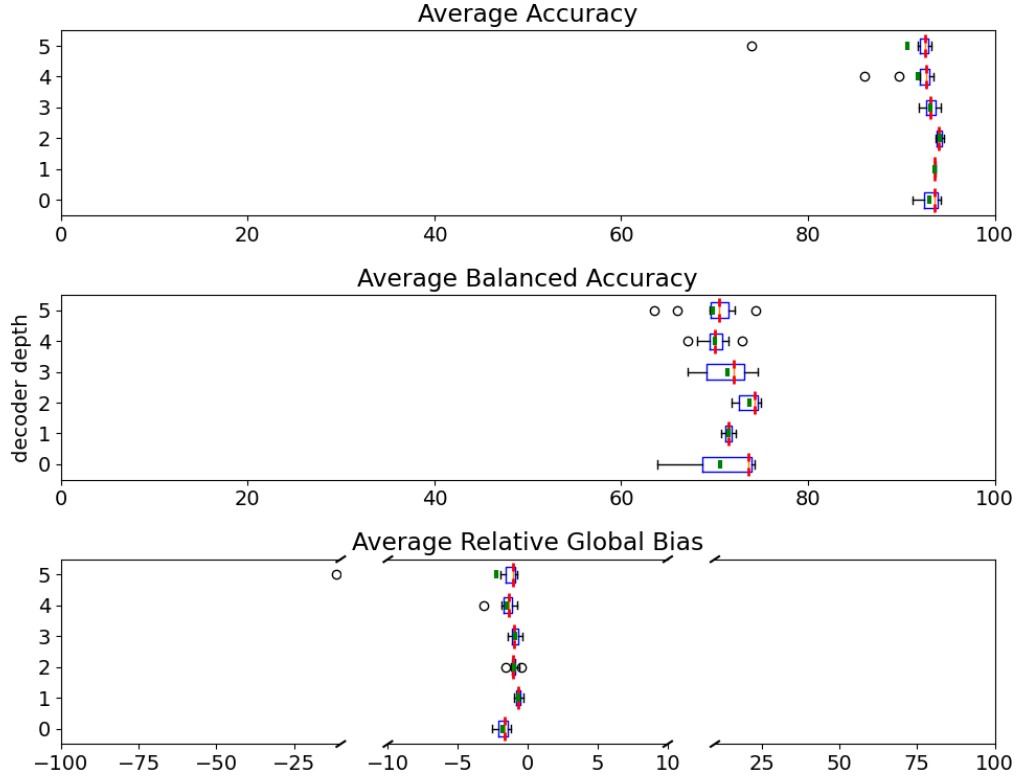


Fig. 5.6: Average Performance in 5 Encoder Blocks Experiment using Dataset B. Five encoder blocks and a varying number of decoder blocks from 0 to 5 were used. Red lines indicate median values, green lines indicate mean values.

As seen with Dataset A, models with 0 and 1 decoder blocks experienced difficulties converging. Only 3 models with 0 decoder blocks and 2 models with 1 decoder block trained for more than 250 epochs.

Figure 5.6 shows that the model with two decoder blocks yielded the highest average accuracy at $94.06\% \pm 0.35\%$, with a balanced accuracy of $73.72\% \pm 1.28\%$ and a relative global bias of $-1\% \pm 0.33\%$.

Insights

The boarding and alighting performance showed similar trends as seen in Section 5.2.1, with boarding accuracy being slightly higher than alighting accuracy.

The consistent challenges faced by models with 0 and 1 decoder blocks, compared to the successful training of the remaining models, suggest that the combination of low spatial dimensions and high feature dimensions in the U-Net output may contribute to instability and slower convergence.

The slight performance improvements observed with fewer decoder blocks suggest that resolving the convergence issues could further enhance the model’s effectiveness.

5.3 U-Net with Batch Normalization

In the previous experiments, instability and convergence issues were observed, particularly with low numbers of decoder blocks. To address this, we introduced batch normalization after every convolutional layer.

Batch normalization helps alleviate the problem of internal covariate shift; that is, as the model trains, the distribution of inputs to each layer changes during training, making it harder for the model to converge and become more unstable. Batch normalization normalizes each convolutional layer output to have a mean of 0 and a variance of 1 for each mini-batch. It then scales and shifts the output through learnable parameters. Batch normalization is a standard method to accelerate convergence, stabilize gradients, and reduce the sensitivity to weight initialization (Ioffe and Szegedy, 2015).

This experiment follows the same configuration as the 3-encoder experiment, with batch normalization added. We chose the 3-encoder configuration in order to enable us to compare all datasets with each other. A 5-encoder configuration using Dataset C would make the image too small at the U-Net base.

5.3.1 Dataset A

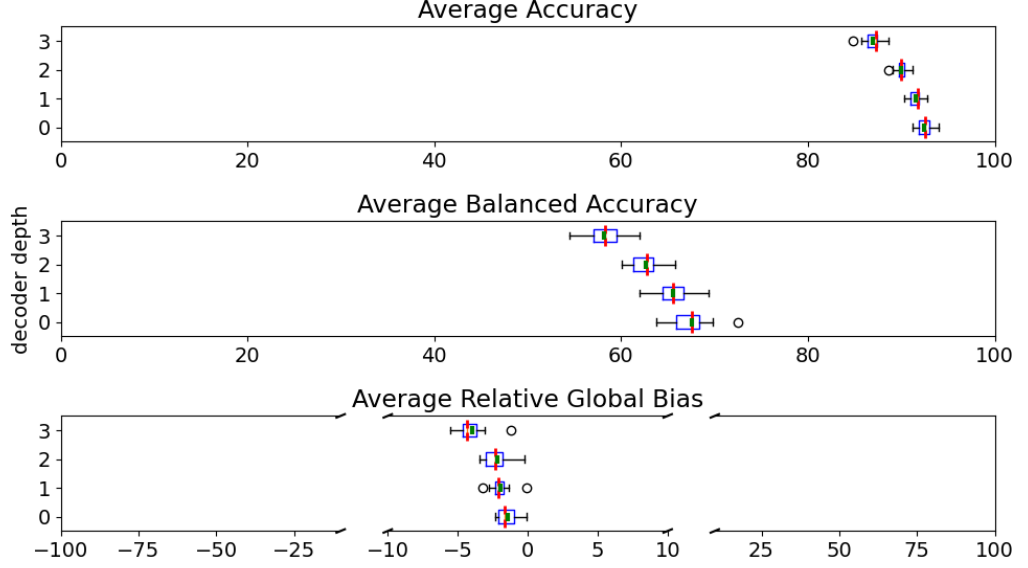


Fig. 5.7: Average Performance in U-Net with Batch Normalisation Experiment using Dataset A. Three encoder blocks and varying from 0 to 3 decoder blocks were used. Red lines indicate median values, green lines indicate mean values.

Figure 5.7 presents the results for Dataset A. Similar to the experiment with three encoder blocks, the relationship between decoder depth and performance remains consistent, with fewer decoder blocks leading to improved performance. However, compared to the non-normalized model, the performance metrics have declined with normalization.

5.3.2 Dataset B

Figure 5.8 presents the results for Dataset B. Unlike the non-normalized model, the models now exhibit a consistent pattern similar to Dataset A, where performance and stability improve as the number of decoder blocks decreases. Although the performance of configurations with three and two decoder blocks declined with the addition of normalization, the average accuracy for the model with zero decoder blocks improved, reaching $93.57\% \pm 0.59\%$.

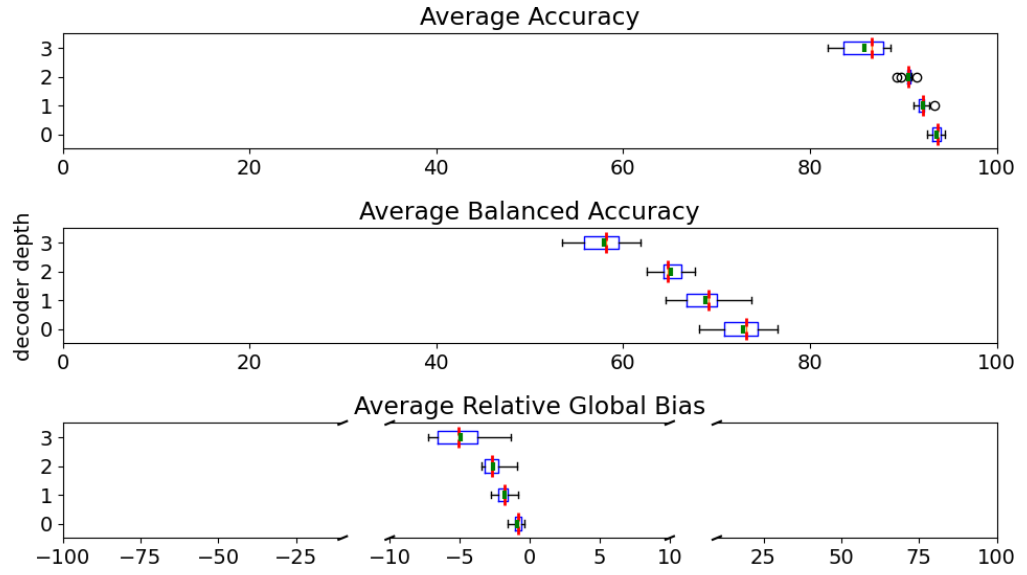


Fig. 5.8: Average Performance in U-Net with Batch Normalisation Experiment using Dataset B. Three encoder blocks and varying from 0 to 3 decoder blocks were used. Red lines indicate median values, green lines indicate mean values.

5.3.3 Dataset C

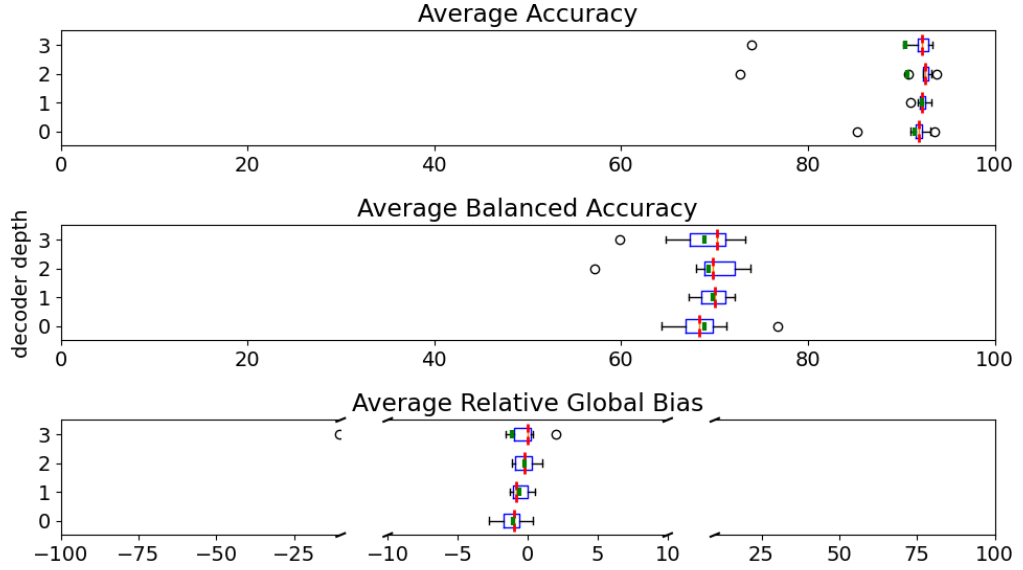


Fig. 5.9: Average Performance in U-Net with Batch Normalisation Experiment using Dataset C. Three encoder blocks and varying from 0 to 3 decoder blocks were used. Red lines indicate median values, green lines indicate mean values.

Figure 5.9 presents the results for Dataset C. Similar to the non-normalized models, all configurations returned very similar results. There is a slight improvement in performance using fewer decoder blocks, as seen in both results for Datasets A and B.

Notably, these models show some of the lowest RGB values out of all experiments, with a total average over all configurations returning an average RGB of $-0.75\% \pm 1.44\%$.

Insights

Batch normalization has reduced the variation in results across all datasets. In all cases, models exhibited at least a slight improvement when fewer decoder blocks

were used. For Datasets A and B, the performance of the models improved compared to their non-normalized counterparts with zero decoder blocks, and they outperformed the models using Dataset C in nearly all performance metrics except for RGB.

Consistent with previous experiments, the models demonstrated better performance in detecting boarding passengers compared to alighting passengers.

5.4 CNN and LSTM

We observed a relationship between fewer decoder blocks and an improved performance throughout some of the experiments. Some of the best-performing models used zero decoder blocks, effectively eliminating the use of skip connections—a core feature of the U-Net architecture. This raises the question of whether the added complexity of the U-Net is necessary for achieving optimal results.

To explore this, we conducted a new experiment using a simplified CNN architecture. The CNN model begins with a Conv3D layer that is immediately followed by a MaxPooling layer. This combination is repeated three times. Each layer uses a 2×2 kernel. The output from the CNN is reshaped and passed through a dense layer before feeding into the LSTM, following the same architecture as the other experiments but replacing the U-Net part.

The results for all three datasets can be seen in figure 5.10. They demonstrated good performance. The models performed slightly worse than the best-performing models with batch normalization, with only a slight decline in all performance measures. This indicates that while U-Nets complexity may contribute to marginally better performance, a simpler CNN architecture can still yield competitive results.

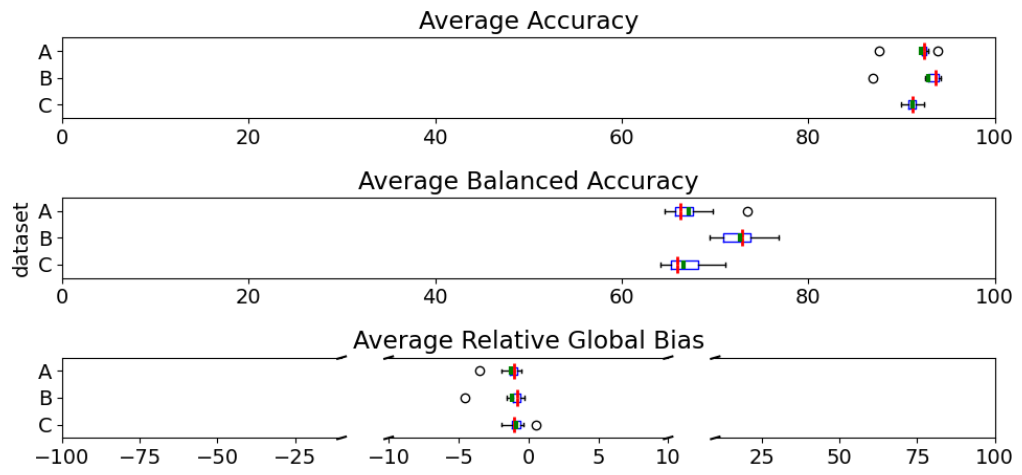


Fig. 5.10: Average Performance in CNN and LSTM Experiment. Red lines indicate median values, green lines indicate mean values.

6 Results

This chapter presents a summary of the experimental insights mentioned in chapter 5, evaluating the performance of the different model configurations tested throughout this study.

6.1 Control Experiment - LSTM Only

The control experiment used only the LSTM model to establish a baseline for comparison. Without convolutional preprocessing, the LSTM-only setup struggled to count passengers accurately, showing high variance in its predictions.

The results underscored the need for effective spatial feature extraction to provide meaningful input to the LSTM. The lack of such preprocessing left the LSTM unable to learn consistently.

6.2 U-Net with Varying Encoder/Decoder Blocks

The next set of experiments introduced U-Net as a preprocessing step, with a varying number of encoder and decoder blocks to explore their influence on model performance.

The findings revealed that reducing the number of decoder blocks consistently led to better accuracy and balanced accuracy across the higher resolution datasets A and B. Notably, models with zero decoder blocks often performed best, indicating that the complexity added by skip connections may not be as critical for this task as initially hypothesized.

Both datasets, A and B, reached their highest average accuracy using asymmetrical U-Net configurations. Dataset C was the only dataset that benefitted from an increased decoder depth.

This suggests that lower-resolution frames benefit from more decoder blocks, as they increase the spatial dimensions before they are passed to LSTM.

6.3 U-Net with Batch Normalization

Batch normalization was added to the U-Net to address issues of instability and convergence, particularly observed in models with five encoder blocks and a low number of decoder blocks. This modification aimed to reduce internal covariate shifts and stabilize the training process.

The results showed that batch normalization helped improve performance for models using fewer decoder blocks. Across all datasets, batch normalization generally reduced the variation in performance, leading to more consistent results. However, performance with batch normalization did not always surpass that of the non-normalized models. This indicates that while batch normalization provided stabilization benefits, it did not always result in significant performance gains.

6.4 Simplified CNN and LSTM Combination

To evaluate whether the complexity of the U-Net was necessary for optimal performance, a simpler CNN architecture was tested in combination with the LSTM.

The simplified CNN model delivered results that were competitive with the U-Net-based models but performed slightly worse across all performance measures. This finding suggests that the added complexity of the U-Net might not be entirely justified. A simpler CNN architecture, which requires fewer computational resources, could potentially serve as a viable alternative without a substantial drop in performance.

6.5 Comparison to other Studies

When comparing the results of this study to previous work on similar APCs, the performance of the models developed here shows promising competitiveness.

In particular, the best model achieved with this study, featuring two decoder blocks, five encoder blocks, and Dataset B without normalization, produced an accuracy of $94.65\% \pm 0.5\%$, a balanced accuracy of $74.03\% \pm 1.04\%$, and a RGB of $-0.89\% \pm 0.44\%$.

This performance is comparable to the work of Jahn (2019), whose model achieved an accuracy of 95.31% and an average RGB of -1.25% .

Results by Shirazi (2021) show similar results. His best comparable model reached an accuracy of $97.4\% \pm 1.2\%$, a balanced accuracy of $72.0\% \pm 0.4\%$, and a RGB of $-0.4\% \pm 0.1\%$.

This shows that despite differences in methodologies, the models developed in this study have the potential to provide similarly accurate passenger counts.

6.6 Additional Observations

Dataset B consistently outperformed both Dataset A and Dataset C across nearly all experiments, suggesting that a more focused cutout combined with high resolution improves model performance.

Throughout all experiments, balanced accuracy was consistently lower than average accuracy, highlighting the challenges associated with class imbalance in the dataset.

All models exhibited a negative RGB, indicating a systematic undercounting of passengers.

Every model performed slightly better when detecting boarding passengers compared to alighting passengers. This discrepancy is likely due to the greater visibility of passengers' faces during boarding.

7 Discussion

The results presented in this study have certain limitations that should be acknowledged.

The number of repetitions for each experimental configuration was limited to a maximum of ten, which restricts the robustness of the conclusions that can be drawn. More extensive testing could provide greater reliability in the findings.

Many models trained for different amounts of epochs due to the 30-hour training time limit, which may have led to variations in the training outcomes and contributed to inconsistencies in performance metrics.

Models were stopped from training after a maximum of 1500 epochs. If longer training time had been allowed, they maybe would have returned better results.

The accuracy and balanced accuracy metrics used in the experiments are more appropriate for binary classification problems, whereas the task at hand is a regression problem. Previous studies working on the same APCS used those metrics (Jahn, 2019) (Shirazi, 2021). In order to remain comparable to them, we used them as well.

Another challenge was the discrepancy between the video content and the provided annotations. Particularly, children under a certain height were excluded from the counts, which could affect the model’s training and evaluation accuracy.

Additionally, the implementation of skip connections in asymmetrical encoder-decoder configurations was an arbitrary choice. In these setups, the decoder block was connected with the encoder block closest in depth, which diverged from the original U-Net design. The original U-Net does not account for such asymmetrical configurations. Although an alternative approach could have been to select different encoder block outputs and scale them to match the corresponding decoder

block, this would also have been an arbitrary decision. The chosen approach in this thesis is one possible method, but it is not standard.

8 Conclusion

This thesis set out to determine whether a combination of U-Net and LSTM can effectively count passengers using 2D video data. We found that LSTM and U-Net architectures work well together, providing a robust foundation for passenger counting in Automatic Passenger Counting Systems.

We discovered, however, that the original U-Net configuration, as presented by Ronneberger et al. (2015), is rarely the most optimal solution for this task. Adjustments, such as varying the encoder and decoder depth, as well as incorporating batch normalization layers, yielded improved results. Additionally, replacing the U-Net component with a simpler CNN demonstrated competitive performance, suggesting that U-Net's complexity is not always necessary. The findings indicate that while LSTM models lack the spatial understanding needed for video analysis, their performance can be substantially enhanced with an appropriate CNN-based feature extraction step.

The specific configuration of the CNN is ultimately dependent on the nature of the task and the dataset. For this study, Dataset B, paired with an asymmetrical U-Net featuring five encoder blocks and two decoder blocks, offered the best results. Further exploration of hyperparameters, including initial filter size, alternative preprocessing strategies, and different CNN or LSTM setups, could potentially lead to an even better-performing model.

Considering the scope and constraints of this thesis, we demonstrated that the methods described are a cost-effective and efficient solution for passenger counting. Many of the models developed achieved a RGB of less than 1%, making them well-suited for equitable revenue distribution among public transport organizations. Depending on the specific requirements of a public transport authority,

8 CONCLUSION

the accuracy we achieved may also suffice for applications related to system load balancing and resource management.

List of References

- Aggarwal, C. C. (2021): Domain-Specific Neural Architectures. In: *Artificial Intelligence: A Textbook*. Cham: Springer International Publishing, 253–298. ISBN: 978-3-030-72357-6. DOI: 10.1007/978-3-030-72357-6_8. URL: https://doi.org/10.1007/978-3-030-72357-6_8 (cit. on pp. 15–20).
- Brodersen, K. H., C. S. Ong, K. E. Stephan, J. M. Buhmann (2010): The Balanced Accuracy and Its Posterior Distribution. *2010 20th International Conference on Pattern Recognition*, 3121–3124. DOI: 10.1109/ICPR.2010.764 (cit. on p. 39).
- Dumoulin, V., F. Visin (2018): *A guide to convolution arithmetic for deep learning*. arXiv: 1603.07285 [stat.ML]. URL: <https://arxiv.org/abs/1603.07285> (cit. on p. 22).
- Dutoit, S. H. C. (2012): *Graphical exploratory data analysis*. <https://worldcat.org/title/1019645745>. Springer. ISBN: 978-1-461-29371-2 (cit. on p. 43).
- Glorot, X., Y. Bengio (2010): Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Y. W. Teh, M. Titterton. Vol. 9. Proceedings of Machine Learning Research. Chia Laguna Resort, Sardinia, Italy: PMLR, 249–256. URL: <https://proceedings.mlr.press/v9/glorot10a.html> (cit. on p. 19).
- Goodfellow, I., Y. Bengio, A. Courville (2016): *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press (cit. on pp. 4–6, 8, 10–12, 14, 19).
- Grgurević, I., K. Juršić, V. Rajič (2022): Review of Automatic Passenger Counting Systems in Public Urban Transport. *5th EAI International Conference on Management of Manufacturing Systems*. Ed. by L. Knapčíková, D. Peraković, A.

LIST OF REFERENCES

- Behúnová, M. Periša. Cham: Springer International Publishing, 1–15. ISBN: 978-3-030-67241-6 (cit. on p. 2).
- Hochreiter, S., J. Schmidhuber (1997): Long Short-term Memory. *Neural computation* 9, 1735–80. DOI: 10.1162/neco.1997.9.8.1735 (cit. on pp. 10, 14).
- Ioffe, S., C. Szegedy (2015): Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *CoRR* abs/1502.03167. arXiv: 1502.03167. URL: <http://arxiv.org/abs/1502.03167> (cit. on p. 51).
- Jahn, N. (2019): *Counting People on Image Sequences Using Recurrent Neural Networks*. Bachelors Thesis (cit. on pp. 35, 59, 60).
- Krizhevsky, A., I. Sutskever, G. E. Hinton (2012): ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*. Ed. by F. Pereira, C. Burges, L. Bottou, K. Weinberger. Vol. 25. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf (cit. on p. 19).
- Land Transport Guru (2018): *Automatic Passenger Counting System for buses*. Accessed: 2024-10-10. URL: <https://landtransportguru.net/automatic-passenger-counting-system/> (cit. on p. 26).
- Nielsen, M. A. (2015): *Neural networks and deep learning*. Vol. 25. Determination press San Francisco, CA, USA (cit. on pp. 4, 5, 8).
- Olah, C. (2015): *Understanding LSTM Networks*. Accessed: 2024-11-21. URL: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/> (cit. on p. 12).
- Pascanu, R., T. Mikolov, Y. Bengio (2013): On the difficulty of training recurrent neural networks. *Proceedings of the 30th International Conference on Machine Learning*. Ed. by S. Dasgupta, D. McAllester. Vol. 28. Proceedings of Machine Learning Research 3. Atlanta, Georgia, USA: PMLR, 1310–1318. URL: <https://proceedings.mlr.press/v28/pascanu13.html> (cit. on p. 11).
- Robinson, T., F. Fallside (1987): *The utility driven dynamic error propagation network* (cit. on p. 8).

LIST OF REFERENCES

- Ronneberger, O., P. Fischer, T. Brox (2015): U-Net: Convolutional Networks for Biomedical Image Segmentation. *CoRR* abs/1505.04597. arXiv: 1505.04597. URL: <http://arxiv.org/abs/1505.04597> (cit. on pp. 22, 23, 62).
- Shirazi, R. D. (2021): Passenger and Object Counting in Sequential Time-of-Flight Sensor Data with Deep Neural Networks. Masters Thesis. *mathesis*. Technical University of Berlin (cit. on pp. 35, 59, 60).
- Sutton, R. S., A. G. Barto (2018): *Reinforcement learning: An introduction*. MIT press (cit. on p. 5).
- Werbos, P. (1990): Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE* 78 (10), 1550–1560. DOI: 10.1109/5.58337 (cit. on p. 8).

Appendix A

Model Performances in Numbers

In this appendix, we present all the results previously discussed in the Results chapter, now displayed as comprehensive tables rather than boxplots. Each table includes the average accuracy (acc), average balanced accuracy (bacc), and average relative global bias (rgb), as well as the performance measures for boarding and alighting.

A.1 LSTM Only

dataset	average acc (%)	average bacc (%)	average rgb (%)
A	80.25 $\pm$ 11.37	53.46 $\pm$ 6.58	-3.23 $\pm$ 4.57
B	83.39 $\pm$ 1.3	55.53 $\pm$ 1.98	-5.03 $\pm$ 1.18
C	88.07 $\pm$ 0.41	61.47 $\pm$ 1.17	-3.04 $\pm$ 0.99

dataset	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
A	81.0 $\pm$ 11.8	54.97 $\pm$ 6.53	-3.46 $\pm$ 3.82
B	83.88 $\pm$ 1.31	55.41 $\pm$ 2.08	-4.72 $\pm$ 1.89
C	88.83 $\pm$ 0.66	61.48 $\pm$ 2.11	-3.06 $\pm$ 1.29

dataset	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
A	79.5 $\pm$ 10.95	51.95 $\pm$ 6.73	-3.0 $\pm$ 5.64
B	82.9 $\pm$ 1.51	55.64 $\pm$ 2.56	-5.35 $\pm$ 1.34
C	87.32 $\pm$ 0.43	61.47 $\pm$ 1.97	-3.03 $\pm$ 1.2

A.2 U-Net with Varying Encoder/Decoder Blocks

A.2.1 3 Encoder Blocks

A.2.1.1 Dataset A

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
3	90.24 ± 1.37	64.0 ± 4.47	-1.37 ± 0.8
2	90.25 ± 2.56	64.93 ± 3.29	-1.54 ± 1.24
1	91.69 ± 2.35	67.82 ± 5.66	-1.6 ± 1.08
0	93.46 ± 1.29	71.59 ± 4.87	-0.92 ± 0.64
decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
3	91.14 ± 1.59	65.11 ± 4.76	-1.42 ± 1.11
2	90.58 ± 4.39	65.29 ± 3.37	-1.62 ± 2.47
1	92.58 ± 2.27	69.09 ± 6.42	-1.53 ± 0.93
0	94.13 ± 0.96	71.76 ± 4.9	-0.86 ± 0.48
decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
3	89.35 ± 2.03	62.89 ± 4.38	-1.32 ± 1.08
2	89.91 ± 1.8	64.57 ± 4.27	-1.46 ± 0.56
1	90.8 ± 2.43	66.54 ± 5.22	-1.67 ± 1.42
0	92.8 ± 1.64	71.43 ± 5.14	-0.98 ± 0.88

APPENDIX A MODEL PERFORMANCES IN NUMBERS

A.2.1.2 Dataset B

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
3	91.85 $\pm$ 1.29	69.83 $\pm$ 1.65	-1.2 $\pm$ 0.3
2	92.56 $\pm$ 0.43	71.19 $\pm$ 1.02	-0.97 $\pm$ 0.46
1	92.23 $\pm$ 1.24	70.51 $\pm$ 3.23	-1.37 $\pm$ 0.82
0	92.42 $\pm$ 1.9	70.98 $\pm$ 2.52	-1.27 $\pm$ 1.32
decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
3	92.58 $\pm$ 1.05	69.76 $\pm$ 1.26	-1.25 $\pm$ 0.68
2	93.27 $\pm$ 0.44	70.78 $\pm$ 1.96	-0.91 $\pm$ 0.54
1	92.8 $\pm$ 1.27	70.67 $\pm$ 3.65	-1.16 $\pm$ 1.02
0	94.04 $\pm$ 0.6	71.45 $\pm$ 2.37	-0.76 $\pm$ 0.42
decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
3	91.12 $\pm$ 1.61	69.9 $\pm$ 2.95	-1.16 $\pm$ 0.74
2	91.85 $\pm$ 0.57	71.6 $\pm$ 1.19	-1.02 $\pm$ 0.58
1	91.66 $\pm$ 1.88	70.35 $\pm$ 3.47	-1.58 $\pm$ 0.98
0	90.8 $\pm$ 3.74	70.5 $\pm$ 3.58	-1.78 $\pm$ 2.34

A.2.1.3 Dataset C

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
3	92.78 $\pm$ 0.57	70.03 $\pm$ 1.82	-0.77 $\pm$ 0.61
2	92.48 $\pm$ 0.77	68.41 $\pm$ 3.02	-0.93 $\pm$ 0.41
1	91.96 $\pm$ 0.84	67.47 $\pm$ 3.19	-1.46 $\pm$ 0.74
0	90.96 $\pm$ 2.27	64.94 $\pm$ 3.6	-1.62 $\pm$ 0.82
decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
3	93.37 $\pm$ 0.52	70.06 $\pm$ 2.29	-0.8 $\pm$ 0.65
2	93.08 $\pm$ 0.67	68.47 $\pm$ 3.88	-0.91 $\pm$ 0.38
1	92.58 $\pm$ 0.69	67.34 $\pm$ 3.07	-1.46 $\pm$ 0.88
0	91.7 $\pm$ 2.11	65.72 $\pm$ 4.07	-1.51 $\pm$ 1.05
decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
3	92.18 $\pm$ 0.74	69.99 $\pm$ 2.42	-0.74 $\pm$ 0.74
2	91.88 $\pm$ 0.98	68.35 $\pm$ 2.9	-0.95 $\pm$ 0.86
1	91.34 $\pm$ 1.12	67.6 $\pm$ 3.65	-1.47 $\pm$ 0.76
0	90.23 $\pm$ 2.45	64.16 $\pm$ 3.37	-1.72 $\pm$ 0.74

A.2.2 5 Encoder Blocks**A.2.2.1 Dataset A**

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
5	90.5 ± 1.59	64.19 ± 3.75	-1.15 ± 0.44
4	90.56 ± 2.45	65.05 ± 4.89	-1.18 ± 1.31
3	91.86 ± 1.91	67.29 ± 4.56	-1.29 ± 0.53
2	90.52 ± 3.52	64.49 ± 6.92	-1.94 ± 2.96
1	92.78 ± 1.83	69.02 ± 4.38	-0.99 ± 0.54
0	89.14 ± 3.26	62.82 ± 5.98	-1.84 ± 0.89

decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
5	91.61 ± 1.28	65.76 ± 3.47	-1.19 ± 0.74
4	90.93 ± 3.93	65.13 ± 5.09	-1.56 ± 2.59
3	92.56 ± 1.76	67.84 ± 5.36	-1.16 ± 0.49
2	91.45 ± 3.59	65.49 ± 6.65	-2.08 ± 3.58
1	93.86 ± 1.03	70.28 ± 3.71	-0.75 ± 0.58
0	90.5 ± 2.92	64.3 ± 6.68	-1.63 ± 0.91

decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
5	89.39 ± 1.98	62.63 ± 4.22	-1.1 ± 0.47
4	90.19 ± 2.18	64.98 ± 5.08	-0.81 ± 0.59
3	91.16 ± 2.08	66.74 ± 4.6	-1.41 ± 0.67
2	89.6 ± 3.57	63.49 ± 7.4	-1.81 ± 2.44
1	91.7 ± 2.63	67.76 ± 5.04	-1.24 ± 0.52
0	87.79 ± 3.6	61.33 ± 5.3	-2.05 ± 1.07

APPENDIX A MODEL PERFORMANCES IN NUMBERS

A.2.2.2 Dataset B

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
5	90.66 $\pm$ 5.91	69.86 $\pm$ 3.1	-2.23 $\pm$ 3.31
4	91.78 $\pm$ 2.24	70.03 $\pm$ 1.63	-1.53 $\pm$ 0.66
3	93.06 $\pm$ 0.77	71.33 $\pm$ 2.5	-0.94 $\pm$ 0.35
2	94.06 $\pm$ 0.35	73.72 $\pm$ 1.28	-1.0 $\pm$ 0.33
1	93.6 $\pm$ 0.06	71.5 $\pm$ 1.08	-0.66 $\pm$ 0.45
0	92.99 $\pm$ 1.57	70.56 $\pm$ 5.8	-1.83 $\pm$ 0.69
decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
5	90.89 $\pm$ 7.41	68.98 $\pm$ 3.32	-2.48 $\pm$ 4.32
4	92.88 $\pm$ 2.03	70.49 $\pm$ 2.26	-1.68 $\pm$ 1.17
3	93.83 $\pm$ 0.69	71.3 $\pm$ 3.06	-0.84 $\pm$ 0.36
2	94.65 $\pm$ 0.5	74.03 $\pm$ 1.4	-0.89 $\pm$ 0.44
1	94.34 $\pm$ 0.13	70.24 $\pm$ 0.21	-0.86 $\pm$ 0.18
0	93.75 $\pm$ 1.5	70.74 $\pm$ 4.89	-1.47 $\pm$ 0.61
decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
5	90.44 $\pm$ 4.43	70.74 $\pm$ 3.62	-1.99 $\pm$ 2.34
4	90.67 $\pm$ 2.83	69.57 $\pm$ 2.79	-1.37 $\pm$ 1.43
3	92.3 $\pm$ 0.88	71.36 $\pm$ 2.28	-1.05 $\pm$ 0.46
2	93.47 $\pm$ 0.26	73.43 $\pm$ 2.19	-1.11 $\pm$ 0.35
1	92.86 $\pm$ 0.25	72.75 $\pm$ 2.38	-0.46 $\pm$ 0.73
0	92.24 $\pm$ 1.64	70.38 $\pm$ 7.04	-2.18 $\pm$ 0.97

A.3 U-Net with Batch Normalisation

A.3.1 Dataset A

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
3	86.91 ± 1.08	58.2 ± 2.18	-3.98 ± 1.2
2	90.01 ± 0.79	62.61 ± 1.71	-2.19 ± 1.03
1	91.57 ± 0.73	65.54 ± 2.12	-1.98 ± 0.85
0	92.45 ± 0.83	67.52 ± 2.5	-1.46 ± 0.76
decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
3	88.28 ± 0.98	59.64 ± 2.8	-3.56 ± 1.73
2	91.26 ± 0.76	64.04 ± 1.96	-1.77 ± 0.81
1	92.49 ± 0.89	66.13 ± 2.29	-1.9 ± 0.79
0	93.32 ± 0.86	69.0 ± 3.63	-1.41 ± 0.44
decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
3	85.54 ± 1.49	56.76 ± 2.54	-4.41 ± 2.32
2	88.76 ± 0.93	61.18 ± 1.85	-2.61 ± 1.53
1	90.64 ± 0.65	64.96 ± 2.47	-2.06 ± 1.31
0	91.59 ± 0.87	66.03 ± 2.3	-1.52 ± 1.17

APPENDIX A MODEL PERFORMANCES IN NUMBERS

A.3.2 Dataset B

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
3	85.87 $\pm$ 2.44	57.91 $\pm$ 2.88	-4.92 $\pm$ 2.03
2	90.49 $\pm$ 0.59	65.06 $\pm$ 1.52	-2.61 $\pm$ 0.74
1	92.09 $\pm$ 0.65	68.77 $\pm$ 2.68	-1.81 $\pm$ 0.61
0	93.57 $\pm$ 0.59	72.86 $\pm$ 2.68	-0.88 $\pm$ 0.35
decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
3	87.04 $\pm$ 2.31	59.17 $\pm$ 3.32	-5.04 $\pm$ 1.75
2	91.2 $\pm$ 0.55	65.07 $\pm$ 1.99	-2.82 $\pm$ 0.52
1	92.91 $\pm$ 0.61	68.62 $\pm$ 3.21	-1.86 $\pm$ 0.74
0	94.29 $\pm$ 0.53	73.48 $\pm$ 2.75	-0.64 $\pm$ 0.42
decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
3	84.71 $\pm$ 2.79	56.65 $\pm$ 3.19	-4.79 $\pm$ 3.03
2	89.78 $\pm$ 0.83	65.06 $\pm$ 1.78	-2.39 $\pm$ 1.47
1	91.28 $\pm$ 0.84	68.91 $\pm$ 4.1	-1.77 $\pm$ 0.7
0	92.86 $\pm$ 0.71	72.23 $\pm$ 3.28	-1.11 $\pm$ 0.58

A.3.3 Dataset C

decoder depth	average acc (%)	average bacc (%)	average rgb (%)
3	90.4 $\pm$ 5.82	68.87 $\pm$ 4.04	-1.12 $\pm$ 3.42
2	90.59 $\pm$ 6.31	69.33 $\pm$ 4.66	-0.25 $\pm$ 0.74
1	92.22 $\pm$ 0.63	69.81 $\pm$ 1.75	-0.57 $\pm$ 0.64
0	91.39 $\pm$ 2.27	68.88 $\pm$ 3.37	-1.08 $\pm$ 0.96
decoder depth	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
3	90.74 $\pm$ 7.36	68.65 $\pm$ 4.55	-1.35 $\pm$ 4.34
2	91.33 $\pm$ 6.65	69.46 $\pm$ 4.21	0.51 $\pm$ 2.13
1	93.12 $\pm$ 0.41	70.89 $\pm$ 2.47	-0.27 $\pm$ 0.8
0	91.4 $\pm$ 4.4	68.56 $\pm$ 5.5	-1.25 $\pm$ 1.89
decoder depth	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
3	90.05 $\pm$ 4.32	69.08 $\pm$ 3.95	-0.89 $\pm$ 2.56
2	89.85 $\pm$ 5.99	69.2 $\pm$ 5.57	-1.02 $\pm$ 3.0
1	91.32 $\pm$ 1.11	68.73 $\pm$ 2.51	-0.88 $\pm$ 0.59
0	91.38 $\pm$ 0.94	69.2 $\pm$ 3.31	-0.9 $\pm$ 1.05

A.4 CNN and LSTM

dataset	average acc (%)	average bacc (%)	average rgb (%)
A	92.08 $\pm$ 1.65	67.19 $\pm$ 2.62	-1.27 $\pm$ 0.89
B	92.88 $\pm$ 2.18	72.68 $\pm$ 2.54	-1.16 $\pm$ 1.25
C	91.16 $\pm$ 0.69	66.61 $\pm$ 2.22	-0.88 $\pm$ 0.67
dataset	boarding acc (%)	boarding bacc (%)	boarding rgb (%)
A	93.31 $\pm$ 0.57	67.54 $\pm$ 3.18	-1.02 $\pm$ 0.59
B	92.81 $\pm$ 4.26	72.16 $\pm$ 3.13	-1.4 $\pm$ 2.44
C	92.06 $\pm$ 0.91	66.97 $\pm$ 2.54	-0.94 $\pm$ 0.75
dataset	alighting acc (%)	alighting bacc (%)	alighting rgb (%)
A	90.85 $\pm$ 3.03	66.85 $\pm$ 2.46	-1.51 $\pm$ 1.55
B	92.95 $\pm$ 0.64	73.21 $\pm$ 2.49	-0.92 $\pm$ 0.56
C	90.25 $\pm$ 0.66	66.25 $\pm$ 2.81	-0.83 $\pm$ 0.97

Appendix B

Failed Experiments

The following models either failed during training or were trained for fewer than 250 epochs and, as a result, were excluded from the experimental performance analysis. If an experiment is not listed here, it successfully ran 10 times for more than 250 epochs.

B.1 U-Net with Varying Encoder/Decoder Blocks

B.1.1 3 Encoder Blocks

B.1.1.1 Dataset A

decoder depth	fails
0	2
1	2
2	1

B.1.1.2 Dataset B

decoder depth	fails
0	1
1	1

B.1.2 5 Encoder Blocks**B.1.2.1 Dataset A**

decoder depth	fails
0	7
1	8
2	2
3	2
4	1
5	2

B.1.2.2 Dataset B

decoder depth	fails
0	7
1	8
2	2